



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр 806

Группа М8О-214СВ-24 Направление подготовки 02.04.02 «Фундаментальная информатика и информационные технологии»

Магистерская программа Информатика

Квалификация: магистр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА МАГИСТРА

На тему: Разработка и исследование пулов потоков с механизмом work-stealing на языке C++

Автор ВКРМ: _____ Спиридонов Кирилл Анатольевич (_____)

Научный руководитель: _____ Морозов Александр Юрьевич (_____)

Консультант: _____ — (_____)

Консультант: _____ — (_____)

Рецензент: _____ — (_____)

К защите допустить

Заведующий кафедрой № 806 _____ Крылов Сергей Сергеевич (_____)

_____ мая 2026 года

Москва 2026

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	4
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	5
ВВЕДЕНИЕ	6
1 Анализ предметной области и обоснование темы исследования . . .	10
1.1 Актуальность проблемы параллельного выполнения задач в современных программных системах	10
1.1.1 Тенденции развития многоядерных и многопроцессорных архитектур	10
1.1.2 Проблемы наивного подхода к управлению потоками . . .	11
1.1.3 Накладные расходы на синхронизацию и балансировку нагрузки как нерешённая практическая проблема	13
1.1.4 Область применения: высоконагруженные серверы, системы реального времени и высокопроизводительные вычисления	15
1.2 Обзор существующих подходов и решений	18
1.2.1 Классические модели управления потоками	18
1.2.2 Механизм work-stealing: история возникновения и теоретические основы	19
1.2.3 Обзор научных и профессиональных публикаций	22
1.2.4 Анализ существующих реализаций пулов потоков	23
1.3 Цели, задачи и требования к разрабатываемому решению	27
1.3.1 Постановка цели исследования	27
1.3.2 Декомпозиция цели на конкретные задачи	28
2 Теоретическое обоснование и проектирование решения	30
2.1 Принципы и логическая модель решения	30
2.1.1 Модель многопоточных вычислений: работа, глубина, критический путь	30
2.1.2 Логика работы классического пула потоков	33
2.1.3 Логика работы пула потоков с механизмом work-stealing .	35
2.2 Архитектура библиотеки и стек технологий	38
2.2.1 Общая архитектура библиотеки	38
2.2.2 Обоснование выбора стека технологий	39
2.2.3 Проектирование программного интерфейса библиотеки . .	40

2.3	Ключевые алгоритмы и структуры данных	42
2.3.1	Алгоритм работы классического пула потоков	42
2.3.2	Двусторонняя очередь WorkStealingDeque	45
2.3.3	Алгоритм работы пула потоков с механизмом work-stealing	48
3	Программная реализация и экспериментальное исследование	53
3.1	Программная реализация библиотеки	53
3.1.1	Структура проекта и организация исходного кода	53
3.1.2	Особенности программной реализации	56
3.2	Данные экспериментального исследования	59
3.2.1	Описание сценариев нагрузки	59
3.2.2	Условия проведения эксперимента	62
3.2.3	Результаты измерений	64
3.3	Тестирование и анализ результатов	67
3.3.1	Тестирование корректности	67
3.3.2	Анализ результатов бенчмарков	68
3.3.3	Обсуждение результатов	71
	ЗАКЛЮЧЕНИЕ	75
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	78
	ПРИЛОЖЕНИЕ А Исходный код	80

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе магистра применяют следующие термины с соответствующими определениями:

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе магистра применяют следующие сокращения и обозначения:

ВВЕДЕНИЕ

На протяжении нескольких десятилетий рост производительности вычислительных систем обеспечивался преимущественно за счёт увеличения тактовой частоты процессоров. Однако в начале 2000-х годов данная тенденция столкнулась с фундаментальными физическими ограничениями, и индустрия перешла к многоядерным архитектурам [1]. Сегодня флагманские серверные процессоры содержат десятки и сотни вычислительных ядер, а параллелизм превратился из узкоспециализированной темы в повседневную инженерную реальность: веб-серверы, игровые движки, системы машинного обучения и базы данных вынуждены явно проектироваться с расчётом на параллельное выполнение.

Ключевым инструментом организации параллельных вычислений в прикладных системах является пул потоков — механизм, при котором фиксированный набор заранее созданных потоков многократно используется для обработки поступающих задач. Вместе с тем классические реализации пулов потоков с единой глобальной очередью задач демонстрируют ограниченную масштабируемость: при увеличении числа рабочих потоков единственная разделяемая очередь превращается в узкое место, порождая конкуренцию и снижая эффективность. Стандартная библиотека языка C++ вплоть до версии C++17 не предоставляет готовой реализации пула потоков [2], что вынуждает разработчиков либо прибегать к крупным внешним зависимостям — таким как Intel Threading Building Blocks, — либо реализовывать собственные решения.

Перспективным подходом к решению проблемы балансировки нагрузки является механизм work-stealing (кражи задач), при котором каждый рабочий поток владеет собственной локальной очередью, а незагруженный поток вправе забирать задачи из очередей более загруженных коллег. Данный механизм обладает строгим теоретическим обоснованием: Блюмофе и Лейзерсон доказали, что жадный алгоритм work-stealing обеспечивает время выполнения $T_P \leq T_1/P + O(T_\infty)$, что является оптимальным с точностью до константного множителя [3]. Несмотря на теоретическую зрелость механизма и его практическое применение в промышленных библиотеках, в открытом доступе отсутствуют реализации, одновременно удовлетворяющие следующим требованиям: построены исключительно на стандарте

C++11/14/17 без внешних зависимостей и подкреплены систематическим экспериментальным исследованием характеристик производительности. Изложенные обстоятельства определяют научную и практическую актуальность настоящей работы.

Цель работы состоит в разработке библиотеки на языке C++, реализующей пул потоков с механизмом work-stealing, и в экспериментальном исследовании её производительности в сравнении с классическим пулом потоков при различных характеристиках нагрузки.

Достижение поставленной цели обеспечивается решением следующих задач:

- а) Проектирование архитектуры библиотеки: разработать модульную архитектуру, включающую классический пул потоков с единой глобальной очередью и пул потоков с механизмом work-stealing, а также единый программный интерфейс, обеспечивающий взаимозаменяемость реализаций.
- б) Реализация двусторонней очереди: разработать структуру данных — двустороннюю очередь (deque) — для механизма work-stealing, обеспечивающую асимметричный доступ владельца и потоков-«воров» с корректной синхронизацией при конкурентном доступе.
- в) Реализация пулов потоков: построить обе реализации пула потоков исключительно на стандартных средствах языка C++11/14/17 без внешних зависимостей в основной библиотеке.
- г) Верификация корректности: разработать автоматические тесты, охватывающие граничные случаи работы реализованных компонентов, и обеспечить отсутствие гонок данных посредством динамического анализа.
- д) Сравнительное исследование производительности: провести измерения производительности обеих реализаций для репрезентативных сценариев нагрузки при различном числе рабочих потоков и сформулировать практические рекомендации по применению.

Объектом исследования являются пулы потоков как механизмы управления параллельным выполнением задач в многоядерных вычислительных системах. Предметом исследования выступают

алгоритмические и реализационные характеристики механизма work-stealing — в части его влияния на масштабируемость, пропускную способность и потребление вычислительных ресурсов — в сравнении с классической архитектурой с единой глобальной очередью.

Теоретические и методические основы работы. Теоретическую основу исследования составляют следующие модели и результаты.

Модель многопоточных вычислений Блюмофе и Лейзерсона [3], представляющая параллельное вычисление в виде ориентированного ациклического графа с параметрами работы T_1 и критического пути T_∞ , используется для теоретического обоснования оптимальности алгоритма work-stealing и формального определения целевых показателей производительности.

Закон Амдала [4] применяется для анализа принципиальных ограничений на масштабируемость параллельных реализаций и обоснования требования к минимизации накладных расходов на синхронизацию.

Алгоритм динамической двусторонней очереди Чейза и Лева [5] служит основой для проектирования структуры данных, обеспечивающей асимметричный доступ владельца и воров.

Методические основы включают: практику параллельного программирования на языке C++ и применение примитивов `std::atomic`, `std::mutex`, `std::condition_variable`, `std::future`, изложенную в книге Уильямса [2]; методы динамического анализа гонок данных посредством Thread Sanitizer; методику статистически корректного измерения производительности с использованием библиотеки Google Benchmark.

Настоящая работа вносит следующие результаты, соответствующие решённым задачам:

- а) Спроектирована модульная библиотека, предоставляющая единый шаблонный интерфейс `submit()` для обеих реализаций пула потоков, что обеспечивает корректность сравнительного исследования при одинаковых условиях.
- б) Реализована двусторонняя очередь `WorkStealingDeque` с асимметричным доступом: владелец работает по принципу LIFO с нижнего конца, потоки-воры — по принципу FIFO с верхнего конца, при корректной синхронизации посредством `std::mutex`.
- в) Реализованы оба пула потоков — `SimpleThreadPool` и

`WorkStealingPool` — без внешних зависимостей, исключительно на средствах стандарта C++17.

- г) Верифицирована корректность всех компонентов посредством 28 автоматических тестов с включённым `Thread Sanitizer`; все тесты успешно пройдены без обнаруженных гонок данных. В ходе тестирования выявлены и устранены нетривиальные дефекты, в том числе гонка данных в деструкторе и переполнение беззнакового типа.
- д) Установлен фундаментальный компромисс между двумя реализациями: `SimpleThreadPool` обеспечивает меньшую латентность реакции на новые задачи (при 16 потоках в сценарии неравномерной нагрузки — 80,3 мс против 98,1 мс), тогда как `WorkStealingPool` потребляет в 3,8 раза меньше процессорного времени (2,75 мс против 10,4 мс), что определяет области предпочтительного применения каждой реализации.

Научная новизна работы заключается в систематическом экспериментальном исследовании влияния механизма ожидания задач на соотношение реального и процессорного времени в реализациях пула потоков с `work-stealing`, а также в выявлении условий, при которых блокирующая синхронизация в структуре двусторонней очереди нивелирует теоретические преимущества механизма кражи задач.

Разработанная библиотека снабжена воспроизводимой системой сборки на основе `CMake` с автоматическим подключением зависимостей через механизм `FetchContent`. Библиотека пригодна для непосредственного встраивания в проекты на языке C++17 в качестве замены `std::async` там, где требуется явное управление числом рабочих потоков.

1 Анализ предметной области и обоснование темы исследования

1.1 Актуальность проблемы параллельного выполнения задач в современных программных системах

1.1.1 Тенденции развития многоядерных и многопроцессорных архитектур

На протяжении нескольких десятилетий основным способом повышения производительности процессоров являлось увеличение тактовой частоты. Данная стратегия позволяла разработчикам программного обеспечения практически бесплатно получать прирост производительности при переходе на новое поколение оборудования. Однако в начале 2000-х годов этот подход столкнулся с фундаментальными физическими ограничениями: дальнейший рост тактовой частоты приводил к недопустимому увеличению тепловыделения и энергопотребления [1].

Ответом индустрии на данное ограничение стал переход к многоядерным архитектурам. Вместо наращивания производительности одного ядра производители начали размещать на одном кристалле несколько вычислительных ядер, каждое из которых работает на относительно невысокой тактовой частоте. Согласно данным компании Intel, уже к 2006 году большинство новых потребительских процессоров выпускалось с двумя и более ядрами, а к 2023 году флагманские серверные процессоры содержат сотни вычислительных ядер [6].

Данная тенденция нашла отражение в известном высказывании Херба Саттера, ставшем крылатой фразой в сообществе разработчиков: «Бесплатный обед закончился» [1]. Суть этого утверждения состоит в том, что простое обновление оборудования более не даёт автоматического прироста производительности для однопоточных программ. Вместо этого разработчики вынуждены явно проектировать свои приложения с расчётом на параллельное выполнение, чтобы задействовать все доступные вычислительные ресурсы.

Параллельно с развитием многоядерных процессоров активно развивались и многопроцессорные системы, в которых несколько физических процессоров объединяются в единую вычислительную платформу. Такие системы получили широкое распространение в области высокопроизводительных вычислений (HPC), центрах обработки данных и

серверных инфраструктурах крупных интернет-компаний. Архитектура NUMA (Non-Uniform Memory Access), характерная для многопроцессорных систем, вносит дополнительную сложность в разработку параллельных программ: время доступа к памяти зависит от того, на каком узле она физически расположена [7].

Таким образом, к середине 2010-х годов параллелизм превратился из узкоспециализированной темы, интересной лишь разработчикам суперкомпьютеров, в повседневную реальность для широкого круга инженеров-программистов. Веб-серверы, игровые движки, системы машинного обучения, базы данных и пользовательские приложения — все они вынуждены эффективно использовать параллелизм для достижения приемлемой производительности [8].

Рост числа ядер порождает принципиально новые требования к механизмам управления параллельным выполнением задач. Простые решения, приемлемые при двух-четырёх ядрах, демонстрируют неудовлетворительное масштабирование при 32, 64 и более ядрах. Именно поэтому исследование эффективных механизмов управления потоками и распределения задач между ними приобретает особую актуальность в современных условиях.

1.1.2 Проблемы наивного подхода к управлению потоками

Наиболее очевидным и простым подходом к параллельному выполнению задач является создание нового потока для каждой поступающей задачи и его уничтожение по завершении работы. Данный подход интуитивно понятен, прост в реализации и нередко используется в учебных примерах и прототипах. Однако при применении в производственных системах он обнаруживает ряд серьёзных недостатков.

Высокие накладные расходы на создание и уничтожение потоков. Создание потока операционной системой является дорогостоящей операцией. В операционной системе Linux создание потока через вызов `clone()` занимает порядка 5–15 мкс [9]. В операционной системе Windows аналогичная операция через `CreateThread()` имеет сопоставимую стоимость. При этом каждый поток требует выделения стека (как правило, от 64 КБ до 8 МБ по умолчанию), инициализации структур данных ядра и регистрации в планировщике операционной системы. Для

высоконагруженных систем, обрабатывающих тысячи и десятки тысяч задач в секунду, совокупные накладные расходы на управление жизненным циклом потоков могут составлять значительную долю общего времени выполнения [10].

Неконтролируемый рост числа потоков. В условиях пиковой нагрузки наивный подход приводит к лавинообразному росту числа одновременно существующих потоков. Поскольку каждый поток потребляет память (прежде всего под стек) и ресурсы ядра операционной системы, бесконтрольное создание потоков способно привести к исчерпанию памяти или достижению системного лимита на количество потоков. В Linux данный лимит определяется параметром ядра `/proc/sys/kernel/threads-max` и по умолчанию составляет порядка нескольких тысяч потоков. Превышение этого лимита влечёт отказ в создании новых потоков и, как следствие, отказ в обслуживании запросов [11].

Избыточное переключение контекста. Планировщик операционной системы распределяет процессорное время между всеми активными потоками посредством вытесняющей многозадачности. Переключение контекста — сохранение состояния текущего потока и восстановление состояния следующего — является затратной операцией, требующей нескольких микросекунд и приводящей к инвалидации кэшей процессора. Когда число активных потоков значительно превышает количество ядер, система тратит существенную долю времени на переключение контекстов вместо полезной работы [12]. Данный эффект получил название *thrashing* и является одной из ключевых проблем наивного подхода.

Отсутствие контроля над использованием ресурсов. Наивный подход не предоставляет механизмов ограничения количества одновременно выполняемых задач. В результате в периоды пиковой нагрузки система оказывается перегруженной, что негативно сказывается на латентности обработки всех запросов, включая приоритетные. Концепция *backpressure* — механизма обратного давления, при котором производитель задач замедляется при переполнении системы — принципиально несовместима с наивным подходом [13].

Неэффективное использование кэша процессора. Потоки, созданные для выполнения коротких задач, как правило, не имеют устойчивой привязки к конкретным ядрам процессора. Планировщик ОС может перемещать поток

между ядрами при каждом пробуждении, что приводит к систематическим промахам кэша и снижению эффективности работы с памятью. В современных процессорах разница в латентности между кэшем L1 и основной памятью достигает 200–300 раз, поэтому влияние данного фактора на производительность весьма существенно [14].

Перечисленные недостатки наивного подхода делают его непригодным для применения в производственных системах с высокими требованиями к производительности и предсказуемости. Это обстоятельство обусловило широкое распространение альтернативной концепции — пула потоков, при которой фиксированный набор заранее созданных потоков многократно используется для выполнения поступающих задач. Однако и в рамках концепции пула потоков остаётся ряд нерешённых проблем, связанных с балансировкой нагрузки между потоками, что будет рассмотрено в последующих разделах.

1.1.3 Накладные расходы на синхронизацию и балансировку нагрузки как нерешённая практическая проблема

Переход к многопоточному программированию неизбежно порождает необходимость координации доступа нескольких потоков к разделяемым ресурсам. Данная координация осуществляется посредством механизмов синхронизации, которые, при всей своей необходимости, сами по себе являются источником существенных накладных расходов и одной из наиболее трудноустраняемых практических проблем в области параллельного программирования [8].

Наиболее распространённым примитивом синхронизации является мьютекс (mutual exclusion lock). Операция захвата свободного мьютекса в Linux на архитектуре x86-64 посредством `pthread_mutex_lock()` занимает порядка 20–40 нс в отсутствие конкуренции. Однако при наличии конкуренции между потоками стоимость возрастает на порядки: поток, не сумевший захватить мьютекс, переводится в состояние ожидания, что влечёт системный вызов, переключение контекста и последующее пробуждение — совокупная стоимость данной цепочки может составлять 1–10 мкс [9]. В высоконагруженных системах, где потоки обращаются к разделяемым ресурсам тысячи раз в секунду, эти задержки накапливаются и становятся определяющим фактором производительности.

Проблема ложного разделения (false sharing). Кэш-память процессора работает с данными не побайтово, а блоками фиксированного размера — кэш-линиями, типичный размер которых составляет 64 байта. Если два потока, выполняющихся на разных ядрах, обращаются к различным переменным, но эти переменные расположены в одной кэш-линии, каждое обновление одной переменной инвалидирует кэш-линию на другом ядре. В результате возникают непроизводительные синхронизации кэша между ядрами, хотя с логической точки зрения потоки работают с независимыми данными [14]. Данное явление, называемое ложным разделением (false sharing), способно снизить производительность параллельной программы до уровня, сопоставимого с однопоточной реализацией или даже хуже неё.

Проблема АВА и сложность неблокирующих алгоритмов. В попытке избежать накладных расходов на блокирующую синхронизацию разработчики прибегают к неблокирующим (lock-free) алгоритмам, основанным на атомарных операциях процессора, в частности на инструкции compare-and-swap (CAS). Однако разработка корректных неблокирующих алгоритмов представляет значительную сложность. Классическим примером является проблема АВА: поток читает значение А, другой поток изменяет его на В, а затем обратно на А, после чего первый поток ошибочно считает, что данные не изменились [15]. Устранение подобных тонких ошибок требует глубокой экспертизы и существенно усложняет реализацию.

Эффективное распределение задач между потоками — балансировка нагрузки — является самостоятельной нетривиальной проблемой. В простейшей модели пула потоков с единой очередью задач все потоки конкурируют за одну разделяемую структуру данных. При большом числе потоков и высокой интенсивности поступления задач данная очередь превращается в узкое место: большую часть времени потоки проводят в ожидании доступа к ней, а не в выполнении полезной работы. Данный эффект получил название contention (конкуренция за ресурс) и является фундаментальным ограничением архитектуры с единой очередью [3].

Для устранения конкуренции применяются архитектуры с отдельными очередями задач для каждого потока. Однако данный подход порождает обратную проблему: при неравномерном поступлении задач одни потоки оказываются перегружены, тогда как другие простаивают в ожидании работы. Неравномерность нагрузки может быть обусловлена как характером

входящих запросов, так и различиями во времени выполнения отдельных задач. Задача оптимальной балансировки нагрузки в реальном времени без значительных накладных расходов на координацию остаётся актуальной научно-практической проблемой [16].

Ещё одной проблемой, связанной с синхронизацией, является инверсия приоритетов: высокоприоритетный поток вынужден ожидать освобождения ресурса, захваченного низкоприоритетным потоком, который, в свою очередь, вытеснен потоком со средним приоритетом. В результате высокоприоритетная задача блокируется на неопределённый срок. Данная проблема приобрела широкую известность после инцидента с марсоходом Mars Pathfinder в 1997 году, когда она привела к систематическим перезагрузкам бортового компьютера [17]. В современных системах реального времени инверсия приоритетов по-прежнему остаётся серьёзной угрозой корректности работы.

Совокупность перечисленных проблем — стоимость синхронизации, ложное разделение, сложность неблокирующих алгоритмов, конкуренция за очередь задач, неравномерность нагрузки и инверсия приоритетов — свидетельствует о том, что построение эффективного механизма управления параллельным выполнением задач является нетривиальной инженерной и исследовательской задачей, не имеющей универсального решения в рамках существующих подходов.

1.1.4 Область применения: высоконагруженные серверы, системы реального времени и высокопроизводительные вычисления

Проблемы эффективного параллельного выполнения задач актуальны для широкого спектра прикладных областей. Ниже рассматриваются наиболее значимые из них с точки зрения требований к механизмам управления потоками.

Современные веб-серверы и серверы приложений обрабатывают сотни тысяч и миллионы запросов в секунду. Такие компании, как Google, Facebook, Amazon и Яндекс, оперируют инфраструктурой, в которой каждый сервер должен поддерживать тысячи одновременных соединений при минимальной латентности отклика [18]. В данном контексте эффективный пул потоков является критически важным компонентом: он определяет пропускную способность сервера и характер деградации под нагрузкой. Неэффективная реализация приводит к резкому росту латентности при приближении к

пиковой нагрузке, что непосредственно влияет на пользовательский опыт и бизнес-показатели.

Характерной особенностью серверных нагрузок является их неоднородность: запросы существенно различаются по сложности и времени обработки, а интенсивность поступления запросов меняется во времени. Это делает статическое распределение задач между потоками неэффективным и требует динамической балансировки нагрузки [19].

Системы реального времени. В системах жёсткого и мягкого реального времени — бортовых авиационных системах, промышленных контроллерах, системах управления транспортными средствами — требования к предсказуемости временных характеристик особенно высоки. Здесь важна не только средняя производительность, но и гарантия того, что задача будет выполнена в пределах установленного временного дедлайна в наихудшем случае (Worst-Case Execution Time, WCET) [20].

Стандартные реализации пулов потоков общего назначения, как правило, не дают подобных гарантий: время ожидания задачи в очереди может варьироваться в широких пределах в зависимости от текущей загрузки системы. Разработка механизмов управления потоками с детерминированными временными характеристиками остаётся открытой исследовательской задачей [21].

Высокопроизводительные вычисления (HPC). В области HPC параллельные вычисления являются основным инструментом решения задач, недоступных для последовательных алгоритмов: молекулярного моделирования, климатических симуляций, задач вычислительной гидродинамики и обучения нейронных сетей [22]. Здесь на первый план выходит эффективность использования вычислительных ресурсов: простой даже небольшой части ядер суперкомпьютера обходится весьма дорого как в денежном, так и во временном отношении.

В HPC-приложениях широко применяется модель разделения задач «разделяй и властвуй» (divide and conquer): крупная задача рекурсивно разбивается на подзадачи вплоть до достижения базового случая. Данная модель естественным образом порождает древовидную структуру зависимостей, в которой динамически создаётся большое количество мелких задач. Именно для данного сценария механизм work-stealing демонстрирует наилучшие теоретические и практические результаты, что будет подробно

рассмотрено в следующем разделе [3].

Таким образом, перечисленные области применения объединяет общая потребность в эффективном, масштабируемом и предсказуемом механизме параллельного выполнения задач. Различия в требованиях — максимальная пропускная способность для серверных систем, предсказуемость латентности для игровых движков, детерминизм для систем реального времени и эффективность использования ресурсов для НРС — определяют многообразие подходов к реализации пулов потоков и обуславливают актуальность их исследования.

1.2 Обзор существующих подходов и решений

1.2.1 Классические модели управления потоками

В процессе развития параллельного программирования сложилось несколько устойчивых моделей управления потоками, каждая из которых представляет собой определённый компромисс между простотой реализации, накладными расходами и эффективностью использования ресурсов.

Исторически первой и наиболее простой является модель «поток на задачу» (thread-per-task), при которой для каждой входящей задачи создаётся отдельный поток, уничтожаемый по завершении работы. Достоинством данной модели является концептуальная простота: разработчику не требуется явно управлять жизненным циклом потоков или координировать распределение задач. Недостатки этой модели — высокие накладные расходы на создание и уничтожение потоков.

Следующим шагом в развитии стала модель пула потоков с единой глобальной очередью (single global queue thread pool). В данной модели при запуске приложения создаётся фиксированное число потоков, равное, как правило, числу логических ядер процессора. Все потоки разделяют одну очередь задач: производитель помещает задачи в конец очереди, а потоки-потребители извлекают их из начала. Данная модель устраняет накладные расходы на создание и уничтожение потоков, однако порождает новую проблему: разделяемая очередь становится точкой конкуренции. При большом числе потоков операции постановки и извлечения задач требуют синхронизации посредством мьютекса или атомарных операций, что ограничивает масштабируемость системы [2].

Для устранения конкуренции за единую очередь была предложена модель пула потоков с локальными очередями (per-thread queue thread pool). В данной архитектуре каждый поток владеет собственной очередью задач. Новые задачи назначаются потокам по некоторому правилу — например, по принципу Round Robin или путём хеширования идентификатора задачи. Данный подход снижает конкуренцию, однако приводит к статическому распределению задач, которое не адаптируется к реальной загрузке потоков. В результате возможна ситуация, когда одни потоки перегружены, тогда как другие простаивают [8].

Попыткой совместить преимущества обеих моделей стала архитектура с

иерархическими очередями (hierarchical queue). В ней глобальная очередь выступает резервуаром задач, а каждый поток дополнительно располагает локальным буфером для хранения небольшого числа задач. Поток в первую очередь берёт задачи из локального буфера, обращаясь к глобальной очереди лишь при его опустошении. Данный подход снижает частоту синхронизации, но не решает проблему неравномерной загрузки потоков [2].

Отдельного рассмотрения заслуживает модель реакторов (reactor pattern) и её расширение — модель SEDA (Staged Event-Driven Architecture). В модели реактора единственный поток обрабатывает события ввода-вывода и диспетчеризирует задачи, что позволяет избежать накладных расходов на синхронизацию, однако не позволяет эффективно задействовать несколько ядер. Модель SEDA разделяет обработку на несколько последовательных стадий, каждая из которых обслуживается собственным пулом потоков, что обеспечивает лучшую масштабируемость и управляемость нагрузки [19].

Анализ перечисленных моделей позволяет выявить фундаментальное противоречие, характерное для классических подходов: снижение конкуренции за разделяемые структуры данных достигается ценой ухудшения балансировки нагрузки, и наоборот. Данное противоречие является основной мотивацией для разработки механизма work-stealing, который стремится одновременно минимизировать синхронизацию и обеспечить динамическую балансировку нагрузки.

1.2.2 Механизм work-stealing: история возникновения и теоретические основы

Механизм work-stealing (кража задач) представляет собой алгоритм динамического распределения задач между потоками, при котором незагруженный поток вправе забирать («красть») задачи из очередей других, более загруженных потоков. Данный подход позволяет совместить низкую конкуренцию за локальные очереди с автоматической балансировкой нагрузки, не требующей централизованного координатора.

Истоки механизма work-stealing восходят к исследованиям в области параллельных вычислений, проводившимся в 1980-е годы. Одной из первых систем, реализовавших схожие принципы, стал язык Multilisp, разработанный Робертом Хэлстедом в Массачусетском технологическом институте в 1985 году [23]. В Multilisp параллельные вычисления организовывались

посредством конструкции `rcall`, а распределение задач между процессорами осуществлялось динамически.

Однако подлинной теоретической основой современного work-stealing стала система Cilk, разработанная в MIT в 1990-е годы группой под руководством Чарльза Лейзерсона. В основополагающей работе Блюмофе и Лейзерсона [3] была формализована модель многопоточных вычислений и доказана оптимальность жадного алгоритма work-stealing с точки зрения теоретической эффективности.

В модели Блюмофе и Лейзерсона многопоточное вычисление представляется в виде ориентированного ациклического графа (DAG), вершины которого соответствуют элементарным инструкциям, а рёбра задают отношения зависимости. Для данной модели вводятся два ключевых параметра: работа T_1 — суммарное число инструкций при выполнении на одном процессоре, и *глубина* (критический путь) T_∞ — длина наидлиннейшей цепочки зависимых инструкций.

Авторы доказали, что при выполнении на P процессорах алгоритм work-stealing обеспечивает время выполнения:

$$T_P \leq \frac{T_1}{P} + O(T_\infty) \quad (1)$$

Данная оценка является оптимальной с точностью до константного множителя: первое слагаемое отражает теоретический минимум, обусловленный объёмом работы, второе — неустранимые потери из-за последовательных зависимостей [3].

Помимо оценки времени выполнения, Блюмофе и Лейзерсон доказали, что алгоритм work-stealing является оптимальным по использованию памяти: потребление памяти при выполнении на P процессорах не превышает P -кратного потребления при последовательном выполнении. Данное свойство принципиально важно для практических приложений, поскольку бесконтрольный рост потребления памяти является типичной проблемой наивных реализаций параллельных вычислений.

Ключевым структурным элементом алгоритма work-stealing является двусторонняя очередь (double-ended queue, deque), которой владеет каждый поток. Работа с данной структурой данных подчиняется следующим правилам:

- поток-владелец помещает новые задачи в нижний конец очереди (push) и извлекает задачи для выполнения также из нижнего конца (pop), то есть работает по принципу стека (LIFO);
- поток-«вор», не имеющий собственных задач, извлекает задачи из верхнего конца очереди жертвы (steal), то есть работает по принципу очереди (FIFO).

Такое асимметричное использование deque преследует две цели. Во-первых, доступ владельца к нижнему концу в большинстве случаев не требует синхронизации, поскольку конкуренция возможна лишь в граничной ситуации, когда очередь содержит ровно одну задачу. Во-вторых, вору забирают задачи с верхнего конца, то есть наиболее «старые» задачи, порождённые ранее всего. В рекурсивных алгоритмах типа «разделяй и властвуй» такие задачи, как правило, являются наиболее крупными и предоставляют больше всего работы, что делает кражу максимально эффективной [16].

Важным практическим достижением стала работа Чейза и Лева [5], в которой была предложена реализация deque для work-stealing на основе динамически расширяемого кольцевого буфера. Данная реализация требует синхронизации (посредством инструкции CAS) лишь в случае конкуренции между владельцем и вором за последнюю задачу в очереди, обеспечивая тем самым практически неблокирующее поведение в типичных сценариях.

После публикации теоретических работ механизм work-stealing был реализован в ряде промышленных систем и языков программирования. Библиотека Intel Threading Building Blocks (TBB), выпущенная в 2006 году, стала одной из первых широко используемых реализаций work-stealing для языка C++ [3]. В языке Java механизм work-stealing был включён в стандартную библиотеку в виде класса ForkJoinPool, появившегося в версии Java 7 в 2011 году. В языке C++ стандарт не предоставляет встроенной реализации work-stealing, что обуславливает актуальность разработки собственных эффективных реализаций [2].

Таким образом, механизм work-stealing сочетает в себе строгое теоретическое обоснование и практическую эффективность, что делает его одним из наиболее перспективных подходов к управлению задачами в параллельных системах.

1.2.3 Обзор научных и профессиональных публикаций

Тематика параллельного программирования и управления потоками получила широкое освещение как в академической литературе, так и в профессиональных технических изданиях. Ниже представлен обзор ключевых работ, формирующих теоретическую и практическую основу настоящего исследования.

Фундаментальной теоретической работой в области work-stealing является статья Блюмофе и Лейзерсона «Scheduling Multithreaded Computations by Work Stealing» [3]. В данной работе авторы формализовали модель многопоточных вычислений на основе ориентированных ациклических графов и доказали, что жадный алгоритм work-stealing обеспечивает оптимальное время выполнения и оптимальное потребление памяти. Результаты этой работы по сей день остаются теоретическим фундаментом для всех практических реализаций механизма кражи задач.

Важное практическое дополнение к теории Блюмофе и Лейзерсона внесли Акар, Блеллох и Блюмофе в работе «The Data Locality of Work Stealing» [16]. Авторы исследовали влияние алгоритма work-stealing на локальность данных в кэш-памяти процессора и показали, что механизм кражи задач обеспечивает хорошую локальность при рекурсивных вычислениях типа «разделяй и властвуй». В частности, было установлено, что поток-владелец, работающий по принципу LIFO, повторно использует данные, недавно помещённые в кэш, тогда как воры, забирающие крупные задачи с верхнего конца очереди, не нарушают эту локальность.

Значительный вклад в практическую реализацию work-stealing внесли Чейз и Лев в работе «Dynamic Circular Work-Stealing Deque» [5]. Авторы предложили реализацию двусторонней очереди на основе динамически расширяемого кольцевого буфера, которая требует синхронизации лишь в редком граничном случае конкуренции за последний элемент. Данная структура данных легла в основу многих промышленных реализаций work-stealing, в том числе в библиотеке Intel TBB.

С точки зрения практики параллельного программирования на языке C++ наиболее полным и авторитетным руководством является книга Энтони Уильямса «C++ Concurrency in Action» [2]. Автор подробно рассматривает средства многопоточности, появившиеся в стандарте C++11 и развитые в

C++14 и C++17: примитивы синхронизации (`std::mutex`, `std::condition_variable`), атомарные операции (`std::atomic`), асинхронное выполнение (`std::future`, `std::async`), а также приводит пример реализации пула потоков с `work-stealing` на основе данных примитивов. Книга Уильямса служит основной методической базой настоящей работы.

Проблематика синхронизации и неблокирующих алгоритмов детально рассмотрена в книге Херлихи и соавторов «The Art of Multiprocessor Programming» [8]. Авторы систематически излагают теорию конкурентных структур данных, доказывают корректность неблокирующих алгоритмов и анализируют их производительность. Особую ценность представляют разделы, посвящённые формальным моделям корректности (линеаризуемости и последовательной согласованности), которые позволяют строго обосновать правильность реализаций `concurrent deque`.

Вопросы операционной поддержки многопоточности рассмотрены в работах Дреппера [9; 14]. В работе «The Native POSIX Thread Library for Linux» описана архитектура библиотеки NPTL, реализующей потоки POSIX в Linux, и анализируются накладные расходы на основные операции. В работе «What Every Programmer Should Know About Memory» детально исследуется влияние иерархии памяти на производительность многопоточных программ, в том числе эффекты ложного разделения и оптимальные стратегии размещения данных в кэш-памяти.

Таким образом, рассмотренные работы охватывают как теоретические основы механизма `work-stealing`, так и практические аспекты его реализации на языке C++. Совокупность данных публикаций формирует достаточную методическую базу для проведения настоящего исследования.

1.2.4 Анализ существующих реализаций пулов потоков

Для обоснования целесообразности разработки собственной реализации пула потоков с механизмом `work-stealing` необходимо проанализировать существующие решения. Ниже рассматриваются наиболее значимые реализации, а также стандартные средства многопоточности языка C++, доступные в стандартах до C++20.

Стандартные средства многопоточности C++ (до C++20). Стандарт C++11 ввёл в язык первую полноценную модель памяти для многопоточных

программ и набор примитивов для работы с потоками [2]. Класс `std::thread` обеспечивает создание и управление потоками операционной системы. Для синхронизации доступны `std::mutex` и его варианты, `std::condition_variable`, а также атомарные типы `std::atomic<T>`. Для асинхронного выполнения задач предусмотрены `std::future`, `std::promise` и функция `std::async`.

Вместе с тем стандартная библиотека C++ (вплоть до C++17 включительно) не предоставляет готовой реализации пула потоков. Функция `std::async` является ближайшим аналогом, однако её поведение существенно ограничено: стандарт не гарантирует повторного использования потоков, а реализация вправе выполнить задачу синхронно [2]. Таким образом, `std::async` не является надёжным инструментом для построения высокопроизводительных параллельных систем и не обеспечивает никакой балансировки нагрузки.

Intel Threading Building Blocks (oneTBB). Библиотека Intel TBB является одной из наиболее зрелых и широко используемых реализаций параллельных вычислений для C++. Она предоставляет высокоуровневые параллельные алгоритмы (`parallel_for`, `parallel_reduce`), конкурентные контейнеры и планировщик задач, основанный на механизме `work-stealing` [3]. Планировщик TBB автоматически определяет число рабочих потоков и управляет их загрузкой.

Ключевым достоинством TBB является высокая производительность и тщательная оптимизация под архитектуры Intel. Среди недостатков следует отметить значительный объём библиотеки, сложность внутреннего устройства, затрудняющую изучение и модификацию, а также историческую привязку к экосистеме Intel, хотя в настоящее время библиотека распространяется как открытый проект oneTBB.

Microsoft Parallel Patterns Library (PPL). Библиотека PPL входит в состав среды разработки Microsoft Visual Studio и предоставляет модель параллельного программирования, концептуально схожую с Intel TBB. PPL включает параллельные алгоритмы (`parallel_for`, `parallel_invoke`), класс `task` для асинхронного выполнения и планировщик `task_scheduler`, также реализующий `work-stealing`.

Существенным ограничением PPL является её привязанность к платформе Windows и среде Visual Studio, что делает невозможным

применение данной библиотеки в кроссплатформенных проектах. Кроме того, в отличие от TBV, PPL не является открытым программным обеспечением, что исключает возможность изучения и модификации её внутреннего устройства.

Пример реализации из книги «C++ Concurrency in Action». Энтони Уильямс в своей книге [2] приводит поэтапную разработку пула потоков с *work-stealing*, построенного исключительно на стандартных средствах C++11/14. Реализация основана на классе `std::thread` для управления рабочими потоками, `std::atomic` для неблокирующей синхронизации и шаблонном классе `work_stealing_queue`, реализующем двустороннюю очередь. Каждый рабочий поток владеет локальной очередью и при её опустошении пытается похитить задачу из очереди случайно выбранного потока.

Данная реализация имеет прежде всего образовательную ценность: она демонстрирует применение стандартных примитивов C++ для построения нетривиальных параллельных структур. Вместе с тем она не претендует на производительность промышленных библиотек и не содержит ряда оптимизаций, характерных для TBV, — в частности, адаптивного управления числом потоков и оптимизации под конкретную иерархию памяти.

Сравнительный анализ рассмотренных реализаций позволяет выявить их ключевые достоинства и недостатки. Существующие решения можно условно разделить на две группы.

Первую группу составляют промышленные библиотеки — Intel TBV и Microsoft PPL. Обе реализуют механизм *work-stealing* и обеспечивают высокую производительность благодаря многолетней оптимизации. Однако Intel TBV представляет собой крупную внешнюю зависимость со сложным внутренним устройством, затрудняющим изучение и модификацию. Microsoft PPL дополнительно ограничена платформой Windows и средой Visual Studio, что исключает её применение в кроссплатформенных проектах. Ни одна из этих библиотек не предоставляет возможности гибкой настройки планировщика под конкретный сценарий нагрузки без глубокого погружения во внутреннее устройство.

Вторую группу образуют стандартные средства C++ и учебные реализации. Функция `std::async`, введённая в стандарте C++11, не имеет внешних зависимостей и доступна на любой платформе, однако не реализует

механизм work-stealing, не гарантирует повторного использования потоков и не предоставляет никаких средств управления балансировкой нагрузки [2]. Реализация пула потоков с work-stealing, приведённая Уильямсом [2], построена исключительно на стандартных примитивах C++11/14, не имеет зависимостей и допускает полный контроль над внутренним устройством. Вместе с тем она носит прежде всего образовательный характер и не содержит оптимизаций, характерных для промышленных библиотек.

Таким образом, в настоящее время отсутствует реализация, которая одновременно удовлетворяла бы следующим требованиям: основана исключительно на стандарте C++11/14, не имеет внешних зависимостей, реализует механизм work-stealing и допускает систематическое исследование влияния параметров на производительность. Разработка такой реализации и её экспериментальное исследование составляют цель настоящей работы.

1.3 Цели, задачи и требования к разрабатываемому решению

1.3.1 Постановка цели исследования

Анализ предметной области, проведённый в разделах 1.1 и 1.2, позволяет сформулировать основное противоречие, определяющее актуальность настоящей работы. С одной стороны, механизм work-stealing обладает строгим теоретическим обоснованием и демонстрирует высокую эффективность в промышленных реализациях. С другой стороны, существующие промышленные реализации (Intel TBB, Microsoft PPL) представляют собой крупные библиотеки со сложным внутренним устройством, не допускающим лёгкой модификации и систематического исследования. Стандартная библиотека C++ (вплоть до C++17) не предоставляет готового пула потоков с механизмом work-stealing, а учебные реализации не подкреплены экспериментальными данными о производительности.

В связи с изложенным целью настоящей выпускной квалификационной работы является разработка библиотеки на языке C++, реализующей пул потоков с механизмом work-stealing, и экспериментальное исследование её производительности в сравнении с классическим пулом потоков при различных сценариях нагрузки.

Поставленная цель предполагает решение как инженерной, так и исследовательской задачи. Инженерная составляющая состоит в проектировании и реализации библиотеки, построенной исключительно на стандартных средствах языка C++ версий C++11/14/17, без внешних зависимостей. Исследовательская составляющая состоит в проведении серии измерений производительности, позволяющих количественно оценить преимущества и ограничения механизма work-stealing при различных условиях применения.

Выбор стандартов C++11/14/17 в качестве технологической основы обусловлен тремя соображениями. Во-первых, все необходимые примитивы многопоточности — `std::thread`, `std::atomic`, `std::mutex`, `std::condition_variable`, `std::future` — были введены в стандарте C++11 и в полной мере доступны начиная с этой версии языка. Во-вторых, стандарты C++11/14/17 по-прежнему остаются основными в большинстве

производственных кодовых баз, что обеспечивает широкую применимость разрабатываемой библиотеки. В-третьих, основная методическая база настоящей работы — книга Уильямса «C++ Concurrency in Action» [2] — рассматривает многопоточность именно в контексте данных стандартов, что обеспечивает строгое соответствие между теоретической основой и практической реализацией.

1.3.2 Декомпозиция цели на конкретные задачи

Для достижения поставленной цели необходимо решить следующие задачи.

Первая задача — анализ существующих подходов и решений. Провести систематический обзор теоретических работ по механизму `work-stealing`, классических моделей управления потоками и существующих реализаций пулов потоков. Выявить достоинства и недостатки существующих решений, обосновать требования к разрабатываемой библиотеке. Данная задача решается в главе 1 настоящей работы.

Вторая задача — проектирование архитектуры библиотеки. Разработать архитектуру библиотеки, включающую две реализации пула потоков: классический пул потоков с единой глобальной очередью задач и пул потоков с механизмом `work-stealing`. Спроектировать единый программный интерфейс, позволяющий использовать обе реализации взаимозаменяемо и проводить их сравнительное исследование в одинаковых условиях.

Третья задача — реализация неблокирующей двусторонней очереди. Разработать и реализовать структуру данных — двустороннюю очередь (`deque`) для механизма `work-stealing`, основанную на атомарных операциях `std::atomic`. Реализация должна обеспечивать неблокирующий доступ владельца очереди и корректную синхронизацию при краже задач сторонними потоками.

Четвёртая задача — реализация пулов потоков. На основе спроектированной архитектуры реализовать классический пул потоков с глобальной очередью и пул потоков с механизмом `work-stealing`. Обе реализации должны быть построены исключительно на стандартных средствах C++11/14/17 и не иметь внешних зависимостей.

Пятая задача — тестирование корректности. Разработать набор тестов, верифицирующих корректность работы реализованных структур данных и

пулов потоков. Тестирование должно охватывать граничные случаи: одновременный доступ к очереди владельца и вора, поведение при пустой очереди, корректность завершения работы пула при наличии незавершённых задач.

Шестая задача — экспериментальное исследование производительности. Разработать методику и набор тестов производительности (benchmarks), позволяющих сравнить классический пул потоков и пул потоков с work-stealing при различных сценариях нагрузки. Исследовать зависимость производительности от числа рабочих потоков, размера и характера задач. На основе полученных данных сформулировать выводы об условиях эффективного применения механизма work-stealing.

Седьмая задача — разработка документации и рекомендаций. Подготовить документацию к разработанной библиотеке и сформулировать практические рекомендации по её применению на основе результатов экспериментального исследования.

В ходе анализа предметной области и обзора существующих решений были получены следующие выводы.

Во-первых, переход к многоядерным архитектурам сделал параллелизм неотъемлемым требованием к современному программному обеспечению. Классические подходы к управлению потоками — модель «поток на задачу» и пул потоков с единой глобальной очередью — обнаруживают существенные ограничения с точки зрения масштабируемости и эффективности балансировки нагрузки при большом числе ядер.

Во-вторых, механизм work-stealing обладает строгим теоретическим обоснованием и доказанной оптимальностью по времени выполнения и потреблению памяти. Вместе с тем существующие промышленные реализации (Intel TBB, Microsoft PPL) не допускают лёгкого изучения и модификации, а стандартная библиотека C++ не предоставляет готовой реализации пула потоков с work-stealing.

В-третьих, сформулированные цель и задачи работы однозначно определяют её границы и критерии успеха: разработать библиотеку на чистом C++11/14/17 без внешних зависимостей, реализующую обе модели пула потоков с единым интерфейсом, и провести их сравнительное экспериментальное исследование.

2 Теоретическое обоснование и проектирование решения

2.1 Принципы и логическая модель решения

2.1.1 Модель многопоточных вычислений: работа, глубина, критический путь

Для теоретического обоснования эффективности механизма work-stealing необходима формальная модель, позволяющая описывать параллельные вычисления и рассуждать об их производительности независимо от конкретной реализации. В данном разделе излагается модель, предложенная Блюмофе и Лейзерсоном [3] и ставшая стандартной теоретической основой для анализа параллельных планировщиков задач.

В модели Блюмофе и Лейзерсона параллельное вычисление представляется в виде ориентированного ациклического графа $G = (V, E)$, где множество вершин V соответствует элементарным вычислительным шагам (инструкциям или задачам), а множество рёбер E задаёт отношения зависимости между ними: ребро $(u, v) \in E$ означает, что задача v не может начать выполнение до завершения задачи u .

Граф вычислений обладает единственной начальной вершиной (исток), соответствующей точке входа в параллельную программу, и единственной конечной вершиной (сток), соответствующей точке её завершения. Вершины, не связанные отношением зависимости, могут выполняться параллельно на различных процессорах. Таким образом, граф G полностью описывает структуру параллелизма вычисления: он определяет, какие задачи могут выполняться одновременно, а какие должны выполняться последовательно.

Рассмотрим в качестве иллюстрации алгоритм параллельного вычисления суммы массива методом «разделяй и властвуй». Массив рекурсивно делится пополам, левая и правая половины суммируются параллельно, после чего результаты складываются. Соответствующий граф вычислений имеет древовидную структуру: каждый внутренний узел соответствует операции сложения двух частичных сумм, а листья — суммированию отдельных элементов. Рёбра графа направлены от листьев к корню, отражая зависимость каждой операции сложения от результатов нижележащих задач.

На основе графа G вводятся два ключевых параметра, определяющих теоретические пределы производительности параллельного вычисления [3].

Работа T_1 — суммарное время выполнения всех вершин графа при использовании одного процессора, то есть длина оптимального последовательного выполнения:

$$T_1 = \sum_{v \in V} w(v), \quad (2)$$

где $w(v)$ — время выполнения задачи v . Работа представляет собой нижнюю оценку суммарного объёма вычислений, который необходимо выполнить независимо от числа процессоров.

Глубина (критический путь) T_∞ — длина наидлиннейшего пути в графе G от истока до стока при неограниченном числе процессоров:

$$T_\infty = \max_{p \in \text{paths}(G)} \sum_{v \in p} w(v), \quad (3)$$

где максимум берётся по всем путям p от истока до стока. Глубина определяет нижнюю оценку времени выполнения вычисления, которую невозможно преодолеть никаким числом процессоров: задачи, лежащие на критическом пути, вынуждены выполняться последовательно в силу отношений зависимости.

Параметры T_1 и T_∞ задают два нижних предела для времени выполнения T_P вычисления на P процессорах [3]:

$$T_P \geq \frac{T_1}{P}, \quad T_P \geq T_\infty. \quad (4)$$

Первое неравенство отражает тот факт, что P процессоров не могут выполнить работу T_1 быстрее, чем за T_1/P . Второе неравенство утверждает, что никакой планировщик не может выполнить вычисление быстрее, чем длина критического пути.

Блюмофе и Лейзерсон доказали, что жадный алгоритм work-stealing обеспечивает время выполнения, близкое к теоретическому минимуму [3]. А именно, при выполнении вычисления с работой T_1 и глубиной T_∞ на P процессорах планировщик work-stealing гарантирует:

$$T_P \leq \frac{T_1}{P} + O(T_\infty). \quad (5)$$

Данная оценка является оптимальной с точностью до константного множителя: первое слагаемое соответствует нижней оценке, обусловленной объёмом работы, второе — нижней оценке, обусловленной глубиной вычисления. Важно отметить, что данная гарантия обеспечивается без какой-либо информации о структуре графа вычислений: планировщик принимает решения о распределении задач динамически, исключительно на основе текущего состояния очередей потоков.

Для количественной оценки эффективности параллельного выполнения вводится понятие ускорения (speedup) — отношения времени последовательного выполнения к времени параллельного выполнения на P процессорах [8]:

$$S_P = \frac{T_1}{T_P}. \quad (6)$$

Идеальным (линейным) ускорением называется случай $S_P = P$, когда использование P процессоров даёт ровно P -кратное ускорение. На практике достичь линейного ускорения невозможно из-за наличия последовательных участков вычисления, накладных расходов на синхронизацию и эффектов иерархии памяти.

Фундаментальное ограничение на достижимое ускорение устанавливает закон Амдала [4]. Пусть доля последовательной части вычисления (то есть части, которая не может быть распараллелена) составляет $\alpha \in [0, 1]$. Тогда максимальное ускорение при использовании P процессоров ограничено сверху:

$$S_P \leq \frac{1}{\alpha + \frac{1-\alpha}{P}}. \quad (7)$$

При $P \rightarrow \infty$ данное выражение стремится к $1/\alpha$, то есть максимальное ускорение ограничено величиной, обратной доле последовательной части. Так, если 10% вычисления является последовательным ($\alpha = 0,1$), то максимально достижимое ускорение не превышает 10 независимо от числа процессоров.

Закон Амдала подчёркивает значимость минимизации последовательных участков в параллельных программах. Именно поэтому накладные расходы на синхронизацию, неизбежно возникающие при использовании разделяемых структур данных, столь критичны для производительности: каждый момент ожидания блокировки фактически увеличивает последовательную долю вычисления и тем самым ограничивает достижимое ускорение.

В контексте разрабатываемой библиотеки закон Амдала определяет ключевое требование к реализации: накладные расходы на управление очередями задач и механизм кражи должны быть минимизированы, чтобы не увеличивать эффективную долю последовательной части вычисления. Именно это требование обуславливает использование атомарных операций вместо блокирующей синхронизации в реализации двусторонней очереди.

2.1.2 Логика работы классического пула потоков

Классический пул потоков реализует модель производитель-потребитель (producer-consumer): один или несколько потоков-производителей формируют задачи и помещают их в общую очередь, тогда как фиксированный набор потоков-потребителей (рабочих потоков) извлекает задачи из очереди и выполняет их [2].

Схема взаимодействия участников классического пула потоков выглядит следующим образом. При инициализации пула создаётся фиксированное число рабочих потоков, как правило равное числу логических ядер процессора. Каждый рабочий поток входит в бесконечный цикл ожидания задач. Поток-производитель помещает задачу в общую очередь и уведомляет об этом один из ожидающих рабочих потоков. Рабочий поток извлекает задачу из очереди, выполняет её и возвращается в состояние ожидания. Доступ к общей очереди со стороны как производителя, так и потребителей защищается мьютексом, что гарантирует корректность при конкурентном доступе [2].

Механизм уведомления рабочих потоков о появлении новых задач реализуется посредством `std::condition_variable`. Рабочий поток, не обнаруживший задач в очереди, вызывает `wait()` на переменной условия и переходит в состояние блокировки, освобождая процессорное время. При поступлении новой задачи производитель вызывает `notify_one()`, что

пробуждает один из заблокированных рабочих потоков. Данный подход позволяет избежать активного ожидания (busy-waiting) и не тратить процессорное время впустую при отсутствии задач [2].

Рассмотрим жизненный цикл задачи в классическом пуле потоков. Он состоит из следующих последовательных этапов.

На первом этапе производитель формирует задачу — вызываемый объект, инкапсулирующий выполняемую работу. Посредством `std::packaged_task` задача связывается с объектом `std::future`, через который производитель впоследствии сможет получить результат выполнения.

На втором этапе производитель захватывает мьютекс, защищающий общую очередь, помещает задачу в её конец, освобождает мьютекс и вызывает `notify_one()` на переменной условия. С момента помещения в очередь задача переходит в состояние ожидания выполнения.

На третьем этапе один из рабочих потоков, пробуждённый уведомлением, захватывает мьютекс, извлекает задачу из начала очереди и освобождает мьютекс. Задача переходит из состояния ожидания в состояние выполнения.

На четвёртом этапе рабочий поток выполняет задачу. Результат выполнения (или исключение, если оно возникло) сохраняется в связанном объекте `std::promise`, что делает его доступным производителю через соответствующий `std::future`.

На пятом этапе рабочий поток возвращается в начало рабочего цикла и вновь пытается извлечь задачу из очереди. Если очередь пуста, поток блокируется на переменной условия до поступления следующей задачи.

Описанная схема обладает рядом достоинств: простотой реализации, предсказуемым поведением и корректностью при любом числе производителей и потребителей. Вместе с тем она имеет существенный недостаток, ограничивающий масштабируемость: единственная разделяемая очередь является точкой конкуренции для всех рабочих потоков. При увеличении числа потоков и интенсивности поступления задач рабочие потоки всё большую долю времени проводят в ожидании мьютекса, а не в выполнении полезной работы [8].

Дополнительным ограничением классической схемы является неэффективность при неравномерной нагрузке. Поскольку задачи

распределяются между потоками в порядке поступления (FIFO), система не может адаптироваться к ситуации, когда одни задачи оказываются значительно длиннее других. В результате часть рабочих потоков может простаивать в ожидании новых задач, тогда как другие потоки заняты выполнением длинных задач. Именно данное ограничение послужило основной мотивацией для разработки механизма work-stealing, который будет рассмотрен в следующем подразделе [2].

2.1.3 Логика работы пула потоков с механизмом work-stealing

Пул потоков с механизмом work-stealing устраняет главное ограничение классической схемы — конкуренцию за единую разделяемую очередь — путём предоставления каждому рабочему потоку собственной локальной очереди задач. При этом сохраняется возможность динамической балансировки нагрузки: поток, исчерпавший собственную очередь, вправе похитить задачу из очереди другого, более загруженного потока [2].

Схема взаимодействия потоков в пуле с work-stealing принципиально отличается от классической модели производитель-потребитель. Каждый рабочий поток одновременно выступает и производителем, и потребителем задач: он порождает подзадачи в ходе выполнения текущей задачи и помещает их в собственную локальную очередь. Внешний производитель при постановке новой задачи направляет её в очередь одного из рабочих потоков — как правило, того, который в данный момент выполняет задачу и, следовательно, имеет наибольшие шансы породить подзадачи в ближайшем будущем [5].

Ключевым структурным элементом данной схемы является двусторонняя очередь (deque), которой владеет каждый рабочий поток. Доступ к deque асимметричен: поток-владелец работает с нижним концом очереди по принципу стека (LIFO), тогда как потоки-воры обращаются к верхнему концу по принципу очереди (FIFO) [3]. Данная асимметрия обеспечивает два важных свойства. Во-первых, операции владельца в типичном случае не требуют блокирующей синхронизации, поскольку конкуренция между владельцем и вором возможна лишь при попытке одновременного доступа к единственному оставшемуся элементу. Во-вторых, вору забирают наиболее «старые» задачи, которые в рекурсивных алгоритмах, как правило, являются наиболее крупными и содержат больше всего потенциальной работы [16].

Рассмотрим жизненный цикл задачи в пуле потоков с work-stealing.

На первом этапе внешний производитель формирует задачу и передаёт её в пул. Задача помещается в локальную очередь одного из рабочих потоков. Если все рабочие потоки в данный момент заняты, задача ожидает в очереди выбранного потока.

На втором этапе рабочий поток, в очередь которого была помещена задача, извлекает её из нижнего конца своей deque и приступает к выполнению. В ходе выполнения задача может порождать подзадачи, которые помещаются в нижний конец той же локальной очереди. Таким образом, подзадачи, порождённые последними, будут выполнены первыми, что соответствует естественному порядку рекурсивного обхода [2].

На третьем этапе, если рабочий поток исчерпал собственную очередь, он становится вором и предпринимает попытку кражи. Поток выбирает жертву — другой рабочий поток — и пытается извлечь задачу с верхнего конца её deque. Если попытка кражи успешна, поток-вор приступает к выполнению похищенной задачи. Если очередь жертвы оказалась пустой, поток предпринимает попытку кражи у другой жертвы.

На четвёртом этапе рабочий поток завершает выполнение задачи, сохраняет результат в связанном объекте `std::promise` и возвращается в начало рабочего цикла.

Алгоритм выбора жертвы для кражи является важным параметром реализации, влияющим как на эффективность балансировки нагрузки, так и на накладные расходы механизма кражи. В литературе описаны несколько подходов к выбору жертвы [3; 2].

Наиболее распространённым является случайный выбор жертвы (random victim selection): поток-вор равномерно случайно выбирает одну из очередей остальных потоков и предпринимает попытку кражи. Данный подход прост в реализации и обеспечивает хорошие теоретические гарантии: Блюмофе и Лейзерсон доказали оценку (5) именно для случайного выбора жертвы [3]. Недостатком является то, что поток может выбрать жертву с пустой или почти пустой очередью, совершив безрезультатную попытку кражи.

Альтернативой является последовательный перебор (round-robin): поток-вор обходит очереди остальных потоков по кругу, начиная со следующего за собой. Данный подход детерминирован и в ряде сценариев

позволяет быстрее обнаружить непустую очередь, однако может приводить к неравномерной нагрузке на отдельные потоки при систематических паттернах постановки задач.

В разрабатываемой библиотеке используется случайный выбор жертвы как наиболее теоретически обоснованный подход. Генерация случайного индекса жертвы осуществляется посредством стандартного генератора `std::mt19937` с равномерным распределением `std::uniform_int_distribution`, что обеспечивает статистическую равномерность выбора без привлечения внешних зависимостей [2].

Важным вопросом является поведение рабочего потока при отсутствии задач во всех очередях пула. В данной ситуации поток может либо продолжать активно опрашивать очереди (spin-waiting), либо перейти в состояние блокировки на переменной условия. Активное ожидание обеспечивает минимальную задержку реакции на появление новой задачи, однако расходует процессорное время. Блокирующее ожидание освобождает процессор, но вносит дополнительную задержку на пробуждение потока [2]. В разрабатываемой библиотеке применяется комбинированный подход: поток выполняет ограниченное число попыток кражи перед переходом в состояние блокировки, что позволяет сбалансировать латентность и потребление ресурсов.

2.2 Архитектура библиотеки и стек технологий

2.2.1 Общая архитектура библиотеки

Разрабатываемая библиотека организована по модульному принципу: каждый компонент несёт единственную ответственность и может быть заменён или расширен независимо от остальных. Библиотека состоит из трёх основных модулей, взаимосвязь которых отражена в структуре заголовочных файлов.

Первый модуль — `WorkStealingDeque` — реализует двустороннюю очередь задач, являющуюся ключевой структурой данных механизма `work-stealing`. Данный модуль не зависит ни от каких других компонентов библиотеки и может использоваться независимо.

Второй модуль — `SimpleThreadPool` — реализует классический пул потоков с единой глобальной очередью задач. Данный модуль использует стандартные примитивы синхронизации C++11: `std::mutex`, `std::condition_variable` и `std::atomic`.

Третий модуль — `WorkStealingPool` — реализует пул потоков с механизмом `work-stealing`. Данный модуль использует `WorkStealingDeque` как внутреннюю структуру данных — по одной очереди на каждый рабочий поток.

Оба класса пулов потоков предоставляют идентичный внешний интерфейс: метод `submit()` для постановки задач и метод `thread_count()` для получения числа рабочих потоков. Идентичность интерфейса обеспечивает взаимозаменяемость реализаций и позволяет проводить корректное сравнительное исследование в одинаковых условиях.

Физическая структура проекта организована следующим образом. Каталог `include/thread_pool/` содержит заголовочные файлы всех трёх модулей. Каталог `src/` содержит файлы реализации для обоих пулов потоков. Каталог `tests/` содержит наборы автоматических тестов корректности. Каталог `benchmarks/` содержит тесты производительности. Такое разделение соответствует общепринятым соглашениям о структуре проектов на языке C++ [2].

2.2.2 Обоснование выбора стека технологий

Выбор инструментов разработки определялся тремя критериями: соответствием стандарту C++17, отсутствием внешних зависимостей в основной библиотеке и наличием развитой экосистемы для тестирования и измерения производительности.

Язык C++ версий C++11/14/17 является основным инструментом разработки. Стандарт C++11 ввёл полноценную модель памяти для многопоточных программ и набор примитивов: `std::thread`, `std::mutex`, `std::condition_variable`, `std::atomic`, `std::future` [2]. Стандарт C++17 добавил `std::invoke_result` и `std::apply`, использованные в реализации метода `submit()`. Все необходимые примитивы доступны в рамках стандартной библиотеки без внешних зависимостей, что обеспечивает максимальную переносимость библиотеки.

Система сборки CMake версии 3.14 и выше выбрана как кроссплатформенный стандарт де-факто для проектов на C++. CMake обеспечивает воспроизводимую сборку на различных платформах и компиляторах, а также предоставляет механизм `FetchContent` для автоматического подключения сторонних зависимостей — библиотек тестирования и измерения производительности.

Библиотека Google Test версии 1.14.0 используется для автоматического тестирования корректности. Выбор обусловлен широкой распространённостью в проектах с открытым исходным кодом, богатым набором макросов для проверки утверждений и встроенной поддержкой в системе сборки CMake через `gtest_discover_tests()`.

Библиотека Google Benchmark версии 1.8.3 используется для измерения производительности. Она обеспечивает статистически корректные измерения: автоматически определяет необходимое число итераций для достижения заданной точности, вычисляет среднее время, стандартное отклонение и позволяет выводить результаты в формате JSON для последующего анализа. Существенным преимуществом является поддержка параметризованных бенчмарков через метод `Range()`, что позволяет автоматически запускать один и тот же сценарий нагрузки с различным числом потоков.

Сравнительная характеристика выбранных инструментов представлена в

таблице 1.

Таблица 1 – Стек технологий разрабатываемой библиотеки

Инструмент	Версия	Назначение
C++	11/14/17	Язык реализации, модель памяти, примитивы многопоточности
CMake	≥ 3.14	Система сборки, управление зависимостями
Google Test	1.14.0	Автоматическое тестирование корректности
Google Benchmark	1.8.3	Измерение и сравнение производительности

2.2.3 Проектирование программного интерфейса библиотеки

Программный интерфейс обоих пулов потоков спроектирован по единому образцу. Центральным элементом интерфейса является шаблонный метод `submit()`, принимающий произвольный вызываемый объект и возвращающий `std::future` с результатом его выполнения:

Листинг 1: Интерфейс метода `submit`

```
1 template<typename F, typename... Args>
2 auto submit(F&& f, Args&&... args)
3     -> std::future<typename std::invoke_result<F, Args...>::type>;
```

Шаблонные параметры `F` и `Args...` позволяют передавать в пул любые вызываемые объекты: свободные функции, лямбда-выражения, объекты с перегруженным `operator()`. Тип возвращаемого значения автоматически выводится посредством `std::invoke_result`, что избавляет пользователя от необходимости явно указывать его.

Возвращаемый объект `std::future` предоставляет три возможности: получить результат выполнения задачи методом `get()` (с блокировкой до завершения), проверить готовность результата без блокировки методом `wait_for()` и получить исключение, если оно возникло в ходе выполнения задачи.

Управление жизненным циклом пула осуществляется через конструктор и деструктор. Конструктор принимает опциональный параметр — число рабочих потоков. Если параметр не указан, используется значение `std::thread::hardware_concurrency()`, соответствующее числу логических ядер процессора. Деструктор обеспечивает корректное завершение: он устанавливает флаг остановки, пробуждает все ожидающие

потоки и дожидается их завершения посредством `std::thread::join()`.

Оба класса объявлены не копируемыми и неперемещаемыми посредством явного удаления соответствующих специальных функций-членов. Данное решение обусловлено тем, что объект пула потоков управляет ресурсами операционной системы (потоками и мьютексами), семантика копирования и перемещения которых неоднозначна.

2.3 Ключевые алгоритмы и структуры данных

2.3.1 Алгоритм работы классического пула потоков

Классический пул потоков построен на трёх взаимодействующих компонентах: очереди задач, защищённой мьютексом, переменной условия для уведомления рабочих потоков и атомарном флаге остановки. Рассмотрим алгоритмы основных операций.

Алгоритм постановки задачи в пул (метод `submit`) представлен в листинге 2.

Листинг 2: Алгоритм постановки задачи в SimpleThreadPool

```
1 template<typename F, typename... Args>
2 auto submit(F&& f, Args&&... args)
3     -> std::future<invoke_result_t<F, Args...>>
4 {
5     using return_type = invoke_result_t<F, Args...>;
6
7     auto task = make_shared<packaged_task<return_type()>>(
8         [f = forward<F>(f),
9         args = make_tuple(forward<Args>(args)...)]
10        () mutable {
11            return apply(move(f), move(args));
12        });
13
14     future<return_type> result = task->get_future();
15
16     {
17         lock_guard<mutex> lock(queue_mutex_);
18
19         if (stop_.load()) {
20             throw runtime_error("Pool is stopped");
21         }
22
23         task_queue_.emplace([task]() { (*task)(); });
24     }
25
26     cv_.notify_one();
27     return result;
28 }
```

Алгоритм рабочего цикла потока состоит из следующих шагов:

- а) Захватить мьютекс очереди задач.
- б) Ждать на переменной условия пока не выполнится предикат: очередь не пуста ($\neg queue.empty()$) или получен сигнал остановки ($stop = true$).
- в) Если получен сигнал остановки и очередь пуста — завершить работу потока.
- г) Извлечь задачу из начала очереди (FIFO).
- д) Освободить мьютекс.
- е) Выполнить задачу.
- ж) Перейти к шагу 1.

Реализация рабочего цикла на языке C++ представлена в листинге 3.

Листинг 3: Рабочий цикл потока SimpleThreadPool

```
1 void SimpleThreadPool::worker_thread()
2 {
3     while (true) {
4         function<void()> task;
5
6         {
7             unique_lock<mutex> lock(queue_mutex_);
8
9             cv_.wait(lock, [this] {
10                 return stop_.load() || !task_queue_.empty();
11             });
12
13             if (stop_.load() && task_queue_.empty()) {
14                 return;
15             }
16
17             task = move(task_queue_.front());
18             task_queue_.pop();
19         }
20
21         task();
22     }
23 }
```

Отметим ключевые свойства данного алгоритма.

Первое свойство — задача извлекается из очереди под защитой мьютекса,

а выполняется вне его. Это принципиально важно: если бы задача выполнялась под мьютексом, другие потоки не могли бы одновременно извлекать задачи из очереди, что свело бы к нулю преимущества параллельного выполнения.

Второе свойство — использование `cv_.wait()` с предикатом защищает от ложных пробуждений. Стандарт C++ допускает что `condition_variable` может пробудить поток без соответствующего уведомления [2]. Предикат гарантирует что поток вернётся в состояние ожидания если условие не выполнено.

Третье свойство — порядок извлечения задач FIFO. Задачи выполняются в том порядке, в котором были поставлены. Это обеспечивает предсказуемое поведение, но не позволяет адаптироваться к неравномерной нагрузке.

Алгоритм корректного завершения работы пула состоит из трёх шагов:

- а) Захватить мьютекс и установить флаг остановки `stop_ = true`. Захват мьютекса гарантирует что рабочий поток либо уже ожидает на переменной условия, либо увидит флаг при следующем захвате мьютекса.
- б) Вызвать `cv_.notify_all()` для пробуждения всех ожидающих потоков.
- в) Вызвать `join()` для каждого рабочего потока, ожидая его завершения.

Реализация деструктора представлена в листинге 4.

Листинг 4: Алгоритм завершения работы SimpleThreadPool

```
1 SimpleThreadPool::~SimpleThreadPool()
2 {
3     {
4         lock_guard<mutex> lock(queue_mutex_);
5         stop_.store(true);
6     }
7
8     cv_.notify_all();
9
10    for (thread& worker : workers_) {
11        if (worker.joinable()) {
12            worker.join();
13        }
14    }
15 }
```

Важным аспектом алгоритма завершения является установка флага `stop_` под защитой мьютекса. Данное решение устраняет гонку данных между деструктором и рабочими потоками. Без захвата мьютекса возможна следующая ситуация: деструктор устанавливает флаг и вызывает `notify_all()`, но рабочий поток ещё не вошёл в `cv_.wait()` — в результате уведомление теряется и поток блокируется навсегда. Захват мьютекса гарантирует что в момент установки флага поток либо уже ожидает на переменной условия (и будет разбужен), либо ещё не захватил мьютекс (и увидит флаг сразу после его захвата) [2].

Рабочие потоки при получении сигнала остановки продолжают выполнять задачи из очереди до её полного опустошения, после чего завершаются. Это обеспечивает корректное завершение (graceful shutdown): все задачи, поставленные до вызова деструктора, будут выполнены.

2.3.2 Двусторонняя очередь `WorkStealingDeque`

Двусторонняя очередь (deque) является ключевой структурой данных механизма work-stealing. Каждый рабочий поток владеет собственным экземпляром данной структуры. Асимметричный доступ к двум концам очереди обеспечивает разделение ролей: владелец работает с одним концом, воры — с противоположным.

Структура `WorkStealingDeque` основана на стандартном контейнере `std::deque<std::function<void()>>`, предоставляющем эффективный доступ к обоим концам последовательности, и мьютексе `std::mutex`, защищающем контейнер от конкурентного доступа. Реализация представлена в листинге 5.

Листинг 5: Структура `WorkStealingDeque`

```
1 class WorkStealingDeque {
2     public:
3         // Только владелец — добавляет в конец (LIFO)
4         void push(std::function<void()> task);
5
6         // Только владелец — извлекает с конца (LIFO)
7         bool pop(std::function<void()>& task);
8
9         // Вор — извлекает с начала (FIFO)
10        bool steal(std::function<void()>& task);
```

```

11
12     std::size_t size() const;
13     bool empty() const;
14
15 private:
16     std::deque<std::function<void()>> queue_;
17     mutable std::mutex mutex_;
18 };

```

Алгоритм операции `push` — добавление задачи владельцем — состоит из следующих шагов:

- а) Захватить мьютекс.
- б) Добавить задачу в конец контейнера (`push_back`).
- в) Освободить мьютекс.

Алгоритм операции `pop` — извлечение задачи владельцем по принципу LIFO — состоит из следующих шагов:

- а) Захватить мьютекс.
- б) Если контейнер пуст — освободить мьютекс и вернуть `false`.
- в) Переместить задачу с конца контейнера (`back`) в выходной параметр.
- г) Удалить последний элемент (`pop_back`).
- д) Освободить мьютекс и вернуть `true`.

Алгоритм операции `steal` — кража задачи вором по принципу FIFO — состоит из следующих шагов:

- а) Захватить мьютекс.
- б) Если контейнер пуст — освободить мьютекс и вернуть `false`.
- в) Переместить задачу с начала контейнера (`front`) в выходной параметр.
- г) Удалить первый элемент (`pop_front`).
- д) Освободить мьютекс и вернуть `true`.

Реализация всех трёх операций представлена в листинге 6.

Листинг 6: Реализация операций `WorkStealingDeque`

```

1 void WorkStealingDeque::push(std::function<void()> task)
2 {
3     std::lock_guard<std::mutex> lock(mutex_);
4     queue_.push_back(std::move(task));
5 }
6

```

```

7 bool WorkStealingDeque::pop(std::function<void()>& task)
8 {
9     std::lock_guard<std::mutex> lock(mutex_);
10
11     if (queue_.empty()) {
12         return false;
13     }
14
15     task = std::move(queue_.back());
16     queue_.pop_back();
17     return true;
18 }
19
20 bool WorkStealingDeque::steal(std::function<void()>& task)
21 {
22     std::lock_guard<std::mutex> lock(mutex_);
23
24     if (queue_.empty()) {
25         return false;
26     }
27
28     task = std::move(queue_.front());
29     queue_.pop_front();
30     return true;
31 }

```

Асимметрия между `pop` и `steal` принципиальна для корректности механизма `work-stealing`. Владелец извлекает задачи с конца очереди (LIFO), что соответствует естественному порядку рекурсивных вычислений: последняя порождённая подзадача выполняется первой, что обеспечивает хорошую локальность данных в кэш-памяти процессора [16]. Воры извлекают задачи с начала очереди (FIFO), то есть наиболее «старые» задачи. В рекурсивных алгоритмах такие задачи, как правило, являются наиболее крупными и содержат наибольший объём потенциальной работы, что делает кражу максимально эффективной [3].

Использование единого мьютекса для всех операций обеспечивает корректность при конкурентном доступе: в любой момент времени не более одного потока выполняет операцию над очередью. Вместе с тем данная реализация не является оптимальной с точки зрения производительности:

мьютекс захватывается при каждой операции, включая операции владельца, которые в более сложных реализациях могут выполняться без синхронизации.

Теоретически оптимальной альтернативой является неблокирующая (lock-free) реализация двусторонней очереди, предложенная Чейзом и Левом [5]. В данной реализации внутреннее хранилище представляет собой динамически расширяемый кольцевой буфер, а индексы нижнего и верхнего концов хранятся в атомарных переменных `std::atomic<std::size_t>`. Ключевым свойством алгоритма Чейза-Лева является то, что операции `push` и `pop` владельца в типичном случае не требуют синхронизации: конкуренция между владельцем и вором возможна лишь при попытке одновременного доступа к единственному оставшемуся элементу, и разрешается посредством атомарной инструкции `compare_exchange_strong` [5].

В рамках настоящей работы была предпринята попытка реализации алгоритма Чейза-Лева, однако в ходе тестирования были обнаружены трудноустранимые ошибки, связанные с тонкими гонками данных при освобождении памяти под задачи. Корректная реализация неблокирующей очереди требует применения техник безопасного освобождения памяти в конкурентной среде — в частности, механизма отложенного удаления (hazard pointers) или подсчёта ссылок на основе атомарных операций. Реализация данных механизмов выходит за рамки настоящей работы, поэтому в качестве рабочего варианта была выбрана более простая реализация на основе мьютекса. Данное решение не влияет на корректность исследования: обе реализации пулов потоков используют одну и ту же структуру `WorkStealingDeque`, что обеспечивает корректность сравнительного эксперимента.

2.3.3 Алгоритм работы пула потоков с механизмом work-stealing

Пул потоков с механизмом work-stealing отличается от классического пула прежде всего алгоритмом рабочего цикла: каждый поток не просто ожидает задачи из общей очереди, а активно пытается найти работу — сначала в собственной очереди, затем в очередях других потоков.

Алгоритм рабочего цикла потока с индексом i состоит из следующих шагов:

- а) Попытаться извлечь задачу из собственной очереди `queues_[i]` методом `pop()` (LIFO). Если задача найдена — выполнить её и

перейти к шагу 1.

- б) Предпринять не более `kMaxStealAttempts = 32` попыток кражи задачи из очереди случайно выбранного потока-жертвы $j \neq i$. Если кража успешна — выполнить задачу и перейти к шагу 1.
- в) Если все попытки кражи безрезультатны — перейти в состояние ожидания на переменной условия на время не более 1 мс.
- г) После пробуждения проверить флаг остановки. Если флаг установлен — выполнить оставшиеся задачи из собственной очереди и завершить работу потока.
- д) Перейти к шагу 1.

Реализация рабочего цикла представлена в листинге 7.

Листинг 7: Рабочий цикл потока `WorkStealingPool`

```
1 void WorkStealingPool::worker_thread(std::size_t index)
2 {
3     current_thread_index_ = index;
4     is_pool_thread_       = true;
5
6     std::mt19937 rng(index);
7     std::uniform_int_distribution<std::size_t> dist(
8         0, workers_.size() - 1);
9
10    static constexpr std::size_t kMaxStealAttempts = 32;
11
12    while (true) {
13        std::function<void()> task;
14
15        // Шаг 1. Берём задачу из своей очереди (LIFO)
16        if (queues_[index]->pop(task)) {
17            task();
18            continue;
19        }
20
21        // Шаг 2. Пытаемся украсть задачу у другого потока
22        bool stolen = false;
23        for (std::size_t attempt = 0;
24            attempt < kMaxStealAttempts && !stolen;
25            ++attempt)
26        {
27            std::size_t victim = dist(rng);
```

```

28         if (victim != index) {
29             stolen = queues_[victim]->steal(task);
30         }
31     }
32
33     if (stolen) {
34         task();
35         continue;
36     }
37
38     // Шаг 3. Задач нет — засыпаем не более чем на 1 мс
39     {
40         std::unique_lock<std::mutex> lock(global_mutex_);
41         cv_.wait_for(
42             lock,
43             std::chrono::milliseconds(1),
44             [this] {
45                 return stop_.load() || any_queue_has_tasks();
46             });
47     }
48
49     // Шаг 4. Проверяем флаг остановки
50     if (stop_.load()) {
51         while (queues_[index]->pop(task)) {
52             task();
53         }
54         return;
55     }
56 }
57 }

```

Алгоритм кражи задач основан на случайном выборе жертвы. Для каждого рабочего потока при его создании инициализируется собственный генератор псевдослучайных чисел `std::mt19937` с начальным значением (seed), равным индексу потока. Использование отдельного генератора для каждого потока исключает конкуренцию за разделяемое состояние генератора и обеспечивает независимость последовательностей выборов жертв у разных потоков.

Равномерное распределение `std::uniform_int_distribution` гарантирует что каждый поток выбирается жертвой с одинаковой

вероятностью $1/(P - 1)$, где P — общее число рабочих потоков. Данное свойство является необходимым условием для теоретических гарантий эффективности алгоритма work-stealing, доказанных Блюмофе и Лейзерсоном [3].

Число попыток кражи перед переходом в состояние ожидания ограничено константой `kMaxStealAttempts = 32`. Данное значение представляет собой компромисс между двумя крайностями. При слишком малом числе попыток поток преждевременно переходит в состояние ожидания и пропускает задачи, появившиеся в очередях других потоков. При слишком большом числе попыток поток расходует процессорное время на безрезультатные попытки кражи из пустых очередей.

Механизм ожидания при отсутствии задач реализован посредством `cv_.wait_for()` с таймаутом 1 мс. Предикат ожидания проверяет наличие задач во всех очередях пула посредством метода `any_queue_has_tasks()`, представленного в листинге 8.

Листинг 8: Проверка наличия задач в очередях пула

```
1 bool WorkStealingPool::any_queue_has_tasks() const
2 {
3     for (const auto& queue : queues_) {
4         if (!queue->empty()) {
5             return true;
6         }
7     }
8     return false;
9 }
```

Таймаут в 1 мс обеспечивает периодическое пробуждение потока даже при отсутствии явного уведомления. Это необходимо потому что предикат `any_queue_has_tasks()` проверяет только текущее состояние очередей: задача может появиться в очереди другого потока после того как данный поток вошёл в состояние ожидания, но до того как был вызван `notify_one()`. В таком случае таймаут гарантирует что поток в конечном счёте обнаружит новую работу.

Данный механизм ожидания определяет фундаментальный компромисс между латентностью реакции на новые задачи и потреблением процессорного времени. Результаты экспериментального исследования, представленные в

главе 3, показывают что `WorkStealingPool` потребляет значительно меньше процессорного времени чем `SimpleThreadPool` при неравномерной нагрузке, однако демонстрирует более высокое реальное время выполнения. Данный эффект объясняется именно механизмом ожидания: потоки `WorkStealingPool` проводят часть времени в состоянии сна, тогда как потоки `SimpleThreadPool` мгновенно реагируют на появление новых задач через `condition_variable`.

В ходе теоретического обоснования и проектирования решения были получены следующие выводы.

Во-первых, формальная модель параллельных вычислений на основе ориентированного ациклического графа позволяет строго обосновать эффективность механизма `work-stealing`. Доказанная Блюмофе и Лейзерсоном оценка $T_P \leq T_1/P + O(T_\infty)$ показывает что алгоритм `work-stealing` обеспечивает время выполнения, близкое к теоретическому минимуму, без каких-либо априорных знаний о структуре вычислений. Закон Амдала определяет ключевое требование к реализации: накладные расходы на управление очередями задач должны быть минимизированы, поскольку они увеличивают эффективную долю последовательной части вычисления и ограничивают достижимое ускорение.

Во-вторых, спроектированная архитектура библиотеки обеспечивает корректность сравнительного исследования. Обе реализации — `SimpleThreadPool` и `WorkStealingPool` — предоставляют идентичный интерфейс метода `submit()`, построены исключительно на стандартных средствах C++11/14/17 и не имеют внешних зависимостей. Это гарантирует что результаты сравнительных бенчмарков отражают именно различия в алгоритмах планирования задач, а не артефакты различных интерфейсов или зависимостей.

В-третьих, выбор упрощённой реализации `WorkStealingDeque` на основе `std::deque` и `std::mutex` вместо неблокирующего алгоритма Чейза-Лева является обоснованным компромиссом для целей настоящей работы. Корректная реализация неблокирующей очереди требует применения механизмов безопасного освобождения памяти в конкурентной среде, реализация которых выходит за рамки исследования. Упрощённая реализация обеспечивает корректность при конкурентном доступе и достаточную производительность для проведения сравнительного эксперимента.

3 Программная реализация и экспериментальное исследование

3.1 Программная реализация библиотеки

3.1.1 Структура проекта и организация исходного кода

Проект организован в соответствии с общепринятыми соглашениями о структуре библиотек на языке C++. Корневой каталог содержит четыре подкаталога, каждый из которых несёт единственную ответственность. Файловая структура проекта представлена в листинге 1.

```
implementation
├── benchmarks
│   └── bench_thread_pools.cpp
├── CMakeLists.txt
├── include
│   └── thread_pool
│       ├── simple_thread_pool.hpp
│       ├── work_stealing_deque.hpp
│       └── work_stealing_pool.hpp
├── src
│   ├── simple_thread_pool.cpp
│   └── work_stealing_pool.cpp
└── tests
    ├── test_simple_pool.cpp
    ├── test_work_stealing_deque.cpp
    └── test_work_stealing_pool.cpp
```

Рисунок 1 – Файловая структура проекта

Каталог `include/thread_pool/` содержит заголовочные файлы всех публичных компонентов библиотеки. Размещение заголовочных файлов в подкаталоге `thread_pool/` обеспечивает пространство имён на уровне файловой системы: пользователь подключает компоненты директивой `#include "thread_pool/simple_thread_pool.hpp"`, что исключает конфликты имён с заголовочными файлами других библиотек.

Каталог `src/` содержит файлы реализации. Разделение интерфейса (заголовочные файлы) и реализации (файлы `.cpp`) является стандартной практикой в C++: оно сокращает время компиляции за счёт раздельной компиляции модулей и скрывает детали реализации от пользователя библиотеки.

Каталог `tests/` содержит автоматические тесты корректности, построенные на основе библиотеки Google Test. Каждый модуль библиотеки имеет соответствующий файл тестов: `test_simple_pool.cpp` тестирует `SimpleThreadPool`, `test_work_stealing_deque.cpp` тестирует `WorkStealingDeque`, `test_work_stealing_pool.cpp` тестирует `WorkStealingPool`.

Каталог `benchmarks/` содержит тесты производительности, построенные на основе библиотеки Google Benchmark. Все сценарии нагрузки сосредоточены в единственном файле `bench_thread_pools.cpp`, что упрощает сравнение результатов двух реализаций.

Система сборки проекта построена на основе CMake версии 3.14 и выше. Файл `CMakeLists.txt` решает четыре задачи: определяет библиотеку `thread_pool` как цель сборки, подключает сторонние зависимости посредством механизма `FetchContent`, определяет исполняемые файлы тестов и бенчмарков и настраивает параметры компиляции.

Механизм `FetchContent` автоматически загружает и собирает библиотеки Google Test и Google Benchmark при первой сборке проекта, что исключает необходимость их ручной установки. Ключевые фрагменты файла `CMakeLists.txt` представлены в листинге 9.

Листинг 9: Ключевые фрагменты `CMakeLists.txt`

```
1 cmake_minimum_required(VERSION 3.14)
2 project(thread_pool VERSION 1.0.0 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7
8 # Основная библиотека — без внешних зависимостей
9 add_library(thread_pool
10     src/simple_thread_pool.cpp
```

```

11     src/work_stealing_pool.cpp
12 )
13 target_include_directories(thread_pool PUBLIC include)
14
15 # Автоматическая загрузка зависимостей
16 include(FetchContent)
17 FetchContent_Declare(googletest ...)
18 FetchContent_Declare(googlebenchmark ...)
19 FetchContent_MakeAvailable(googletest googlebenchmark)
20
21 # Тесты — с ThreadSanitizer для обнаружения гонок данных
22 add_executable(tests ...)
23 target_compile_options(tests PRIVATE -fsanitize=thread)
24 target_link_options(tests PRIVATE -fsanitize=thread)
25
26 # Бенчмарки — с оптимизацией O3
27 add_executable(benchmarks ...)
28 target_compile_options(benchmarks PRIVATE -O3)

```

Тесты корректности собираются с включённым Thread Sanitizer (`-fsanitize=thread`) — инструментом динамического анализа, обнаруживающим гонки данных во время выполнения программы. Бенчмарки производительности собираются с уровнем оптимизации `-O2` и без санитайзеров, поскольку санитайзеры вносят значительные накладные расходы и искажают результаты измерений.

Сборка и запуск проекта осуществляются следующей последовательностью команд, представленной в листинге 10.

Листинг 10: Сборка и запуск проекта

```

1 # Конфигурация в режиме Release
2 cmake -B build -DCMAKE_BUILD_TYPE=Release
3
4 # Сборка всех целей
5 cmake --build build
6
7 # Запуск тестов корректности
8 ctest --test-dir build --output-on-failure
9
10 # Запуск бенчмарков с выводом в JSON
11 ./build/benchmarks \
12     --benchmark_format=json \

```

3.1.2 Особенности программной реализации

В ходе практической реализации библиотеки был выявлен ряд нетривиальных проблем, решение которых потребовало отклонения от первоначального проектного решения. Данный раздел описывает наиболее значимые из них.

Первая проблема — гонка данных в деструкторе `SimpleThreadPool`. В исходной реализации деструктор устанавливал флаг остановки и вызывал `notify_all()` без захвата мьютекса:

Листинг 11: Исходная некорректная реализация деструктора

```
1 // Некорректно: возможна потеря уведомления
2 SimpleThreadPool::~SimpleThreadPool() {
3     stop_.store(true);           // флаг установлен
4     cv_.notify_all();           // уведомление отправлено
5     for (auto& w : workers_) w.join();
6 }
```

При данной реализации возникала следующая гонка: рабочий поток мог проверить предикат переменной условия, не обнаружить задач, затем деструктор устанавливал флаг и отправлял уведомление, и лишь после этого поток входил в `cv_.wait()`. В результате уведомление было потеряно и поток блокировался навсегда. Проблема была обнаружена при стресс-тестировании: тест `ThreadCountIsCorrect` периодически зависал при запуске совместно с другими тестами.

Решение состоит в захвате мьютекса перед установкой флага остановки, что гарантирует атомарность операции с точки зрения рабочих потоков:

Листинг 12: Исправленная реализация деструктора

```
1 SimpleThreadPool::~SimpleThreadPool() {
2     {
3         // Захват мьютекса гарантирует что поток
4         // либо уже в cv_.wait(), либо увидит флаг
5         // при следующем захвате мьютекса
6         std::lock_guard<std::mutex> lock(queue_mutex_);
7         stop_.store(true);
8     }
```



```

9     cv_.notify_all();
10    for (auto& w : workers_) w.join();
11 }

```

Вторая проблема — переполнение беззнакового типа в методе `pop()` класса `WorkStealingDeque`. В исходной реализации алгоритма Чейза-Лева метод `pop()` первым действием уменьшал значение `bottom_` на единицу:

Листинг 13: Ошибка переполнения в методе `pop()`

```

1 bool pop(std::function<void()>& task) {
2     // Ошибка: если bottom_ == 0, то bottom_ - 1
3     // равно 18446744073709551615 переполнение( size_t)
4     std::size_t bottom =
5         bottom_.load(std::memory_order_relaxed) - 1;
6     ...
7 }

```

При вызове `pop()` на пустой очереди, когда `bottom_ = 0`, беззнаковое вычитание приводило к переполнению и обращению к недопустимой области памяти, что вызывало аварийное завершение программы (segmentation fault). Проблема была обнаружена тестом `PopOnEmptyReturnsFalse`.

Решение состоит в проверке равенства `bottom_` и `top_` до выполнения каких-либо изменений:

Листинг 14: Исправленный метод `pop()`

```

1 bool pop(std::function<void()>& task) {
2     std::size_t bottom = bottom_.load(std::memory_order_relaxed);
3     std::size_t top     = top_.load(std::memory_order_acquire);
4
5     // Проверяем до вычитания — исключаем переполнение
6     if (bottom == top) {
7         return false;
8     }
9
10    bottom -= 1;
11    ...
12 }

```

Третья проблема — отказ от lock-free реализации `WorkStealingDeque`. Первоначально планировалась реализация неблокирующей двусторонней очереди по алгоритму Чейза-Лева [5] на основе

атомарных операций `std::atomic`. Однако в ходе тестирования были обнаружены ошибки типа `double free` и повреждение кучи (`malloc_consolidate(): invalid chunk size`), связанные с гонкой данных при освобождении памяти под задачи.

Корень проблемы состоит в том что вор читает указатель на задачу из буфера до выполнения операции CAS. В промежутке между чтением и CAS владелец может выполнить `pop()` и удалить задачу, оставив у вора висячий указатель. Корректное решение требует применения механизмов безопасного освобождения памяти в конкурентной среде — `hazard pointers` или эпохального удаления (`epoch-based reclamation`), реализация которых существенно выходит за рамки настоящей работы.

В качестве рабочего варианта была выбрана реализация на основе `std::deque` и `std::mutex`, корректность которой очевидна и легко верифицируется. Данное решение не влияет на корректность сравнительного исследования, поскольку обе реализации пулов потоков используют одну и ту же структуру `WorkStealingDeque`.

Для обнаружения гонок данных в ходе разработки использовался инструмент Thread Sanitizer (TSan) — динамический анализатор, встроенный в компиляторы GCC и Clang. TSan инструментирует код во время компиляции и обнаруживает гонки данных во время выполнения программы. Тесты корректности собираются с флагами `-fsanitize=thread`, что позволяет автоматически проверять отсутствие гонок данных при каждом запуске тестов. В финальной версии библиотеки Thread Sanitizer не обнаружил ни одной гонки данных.

3.2 Данные экспериментального исследования

3.2.1 Описание сценариев нагрузки

Для сравнительного исследования производительности двух реализаций пула потоков были разработаны два сценария нагрузки, отражающих типичные паттерны использования пулов потоков в производственных системах. Каждый сценарий реализован в виде параметризованного бенчмарка библиотеки Google Benchmark, запускаемого с числом рабочих потоков 1, 2, 4, 8 и 16.

Первый сценарий — много мелких задач — моделирует ситуацию, характерную для высоконагруженных серверных систем, в которых пул потоков обрабатывает большое число коротких независимых запросов. В данном сценарии в пул последовательно ставятся 10 000 задач, каждая из которых выполняет атомарный инкремент счётчика. Атомарный инкремент выбран намеренно: его время выполнения составляет единицы наносекунд, что обеспечивает доминирование накладных расходов планировщика над полезной работой. Таким образом, данный сценарий измеряет прежде всего эффективность механизма постановки и извлечения задач, а не производительность самих задач. Реализация сценария представлена в листинге 15.

Листинг 15: Сценарий 1: много мелких задач

```
1 static constexpr int kSmallTaskCount = 100000;
2
3 static void BM_SimplePool_ManySmallTasks(
4     benchmark::State& state)
5 {
6     const std::size_t thread_count =
7         static_cast<std::size_t>(state.range(0));
8
9     SimpleThreadPool pool(thread_count);
10
11     for (auto _ : state) {
12         std::atomic<int> counter(0);
13         std::vector<std::future<void>> futures;
14         futures.reserve(kSmallTaskCount);
15
16         for (int i = 0; i < kSmallTaskCount; ++i) {
17             futures.emplace_back(pool.submit([&counter]() {
```

```

18         counter.fetch_add(1,
19             std::memory_order_relaxed);
20     }));
21 }
22
23 for (auto& f : futures) { f.get(); }
24
25 if (counter.load() != kSmallTaskCount) {
26     state.SkipWithError("Not all tasks completed");
27 }
28 }
29
30 state.SetItemsProcessed(
31     state.iterations() * kSmallTaskCount);
32 }

```

Для данного сценария ожидается следующее поведение. При малом числе потоков обе реализации должны демонстрировать схожую производительность, поскольку конкуренция за разделяемые структуры данных невелика. При увеличении числа потоков SimpleThreadPool должен деградировать быстрее из-за конкуренции всех потоков за единственную глобальную очередь. WorkStealingPool теоретически должен масштабироваться лучше, однако использование мьютекса в WorkStealingDeque ограничивает это преимущество.

Второй сценарий — неравномерная нагрузка — моделирует ситуацию, в которой задачи существенно различаются по времени выполнения. Данный паттерн характерен для систем обработки запросов, где часть запросов требует обращения к базе данных или внешним сервисам, а другая часть обрабатывается локально. В данном сценарии в пул ставятся 10 000 задач, половина из которых является короткими (атомарный инкремент), а другая половина — длинными (задержка 200 мкс посредством `std::this_thread::sleep_for`). Короткие и длинные задачи чередуются: чётные по индексу задачи являются длинными, нечётные — короткими. Реализация сценария представлена в листинге 16.

Листинг 16: Сценарий 2: неравномерная нагрузка

```

1 static constexpr auto kLongTaskDelay =
2     std::chrono::microseconds(200);
3

```

```

4 static void BM_SimplePool_UnevenLoad(
5     benchmark::State& state)
6 {
7     const std::size_t thread_count =
8         static_cast<std::size_t>(state.range(0));
9
10    SimpleThreadPool pool(thread_count);
11
12    for (auto _ : state) {
13        std::atomic<int> counter(0);
14        std::vector<std::future<void>> futures;
15        futures.reserve(kSmallTaskCount);
16
17        for (int i = 0; i < kSmallTaskCount; ++i) {
18            futures.emplace_back(
19                pool.submit([&counter, i]() {
20                    if (i % 2 == 0) {
21                        std::this_thread::sleep_for(
22                            kLongTaskDelay);
23                    }
24                    counter.fetch_add(1,
25                        std::memory_order_relaxed);
26                }));
27        }
28
29        for (auto& f : futures) { f.get(); }
30
31        if (counter.load() != kSmallTaskCount) {
32            state.SkipWithError("Not all tasks completed");
33        }
34    }
35
36    state.SetItemsProcessed(
37        state.iterations() * kSmallTaskCount);
38 }

```

Для данного сценария ожидается что WorkStealingPool продемонстрирует преимущество перед SimpleThreadPool при большом числе потоков. Механизм кражи задач позволяет незагруженным потокам забирать задачи у потоков, занятых выполнением длинных задач, что должно обеспечить более равномерную загрузку всех ядер процессора.

Выбор данных двух сценариев обусловлен тем, что они охватывают два полярных случая с точки зрения балансировки нагрузки. В первом сценарии все задачи одинаковы по длительности — балансировка нагрузки не требуется и оба пула должны работать схожим образом. Во втором сценарии задачи существенно различаются по длительности — именно здесь механизм work-stealing имеет теоретическое преимущество. Сравнение результатов двух сценариев позволяет выявить условия, при которых механизм work-stealing оправдывает свои накладные расходы.

Параметры бенчмарков представлены в таблице 2.

Таблица 2 – Параметры экспериментального исследования

Параметр	Значение
Число задач в сценарии	10 000
Длительность короткой задачи	Атомарный инкремент (≈ 5 нс)
Длительность длинной задачи	<code>sleep_for(200 мкс)</code>
Исследуемые значения числа потоков	1, 2, 4, 8, 16
Число итераций	Определяется автоматически библиотекой Google Benchmark
Режим измерения времени	Реальное время (UseRealTime)
Уровень оптимизации компилятора	-O2

3.2.2 Условия проведения эксперимента

В данном разделе описываются характеристики тестовой машины и параметры запуска бенчмарков.

Эксперимент проводился на рабочей станции под управлением операционной системы Linux. Характеристики аппаратного обеспечения были получены непосредственно из вывода библиотеки Google Benchmark и представлены в таблице 3.

Следует отметить что при проведении эксперимента была включена динамическая регулировка тактовой частоты процессора (CPU frequency scaling). Google Benchmark выводит предупреждение о данном факте:

Листинг 17: Предупреждение Google Benchmark о CPU scaling

```

1 ***WARNING*** CPU scaling is enabled, the benchmark real
2 time measurements may be noisy and will incur extra overhead.
```

Динамическая регулировка тактовой частоты означает что процессор

Таблица 3 – Характеристики тестовой машины

Параметр	Значение
Число логических ядер	16
Тактовая частота	5102,71 МГц
Кэш L1 (данные)	32 КиБ × 8
Кэш L2	1024 КиБ × 8
Кэш L3	16384 КиБ × 1
Операционная система	Linux
Компилятор	Clang, стандарт C++17
Уровень оптимизации	-O2

может изменять частоту в ходе эксперимента в зависимости от тепловой нагрузки и энергопотребления. Это вносит дополнительный разброс в результаты измерений. Отключение данной функции требует прав администратора и изменения системных параметров ядра Linux, что не всегда возможно в стандартной рабочей среде. Для компенсации данного эффекта Google Benchmark автоматически увеличивает число итераций до достижения статистически значимых результатов.

Параметры запуска бенчмарков определялись следующим образом. Число итераций для каждого бенчмарка определялось автоматически библиотекой Google Benchmark: библиотека увеличивает число итераций до тех пор пока суммарное время выполнения не превысит пороговое значение, обеспечивающее статистически значимый результат. Для каждого бенчмарка измерялось реальное время выполнения (`real_time`), а не процессорное время (`cpu_time`), поскольку реальное время отражает фактическую пропускную способность системы с учётом времени ожидания потоков.

Исследуемые значения числа потоков — 1, 2, 4, 8 и 16 — выбраны как степени двойки в диапазоне от одного потока до полного использования всех логических ядер тестовой машины. Данный диапазон позволяет исследовать три характерных режима работы пула потоков. При числе потоков меньше числа ядер (1, 2, 4) система недозагружена и накладные расходы планировщика минимальны. При числе потоков равном числу ядер (16) достигается теоретически оптимальная загрузка. Промежуточное значение 8 позволяет наблюдать переходный режим.

Каждый бенчмарк запускался в рамках единственного процесса без параллельного выполнения других бенчмарков, что исключает взаимное

влияние сценариев нагрузки на результаты измерений. Бенчмарки и тесты корректности запускались отдельно: тесты собираются с Thread Sanitizer, который вносит значительные накладные расходы и не должен влиять на измерения производительности.

3.2.3 Результаты измерений

В данном разделе представлены результаты измерений производительности обеих реализаций пула потоков для двух сценариев нагрузки. Измерения проводились посредством библиотеки Google Benchmark. Для каждой конфигурации фиксировались три метрики: реальное время выполнения одной итерации бенчмарка (Time), суммарное процессорное время потоков (CPU) и пропускная способность в задачах в секунду (items/s).

Результаты для сценария 1 (много мелких задач) представлены в таблице 4.

Таблица 4 – Результаты сценария 1: много мелких задач (10 000 задач, атомарный инкремент)

Потоков	Реализация	Time, мс	CPU, мс	items/s
1	SimpleThreadPool	4,52	3,60	2 211 780
1	WorkStealingPool	4,39	3,77	2 275 520
2	SimpleThreadPool	3,76	3,30	2 659 860
2	WorkStealingPool	4,86	4,26	2 055 800
4	SimpleThreadPool	6,77	5,56	1 476 680
4	WorkStealingPool	7,93	6,52	1 260 560
8	SimpleThreadPool	14,2	10,5	702 662
8	WorkStealingPool	14,8	11,5	677 055
16	SimpleThreadPool	12,5	10,3	802 883
16	WorkStealingPool	15,2	13,3	656 266

По результатам измерений можно выделить несколько наблюдений.

В сценарии 1 обе реализации демонстрируют схожую тенденцию: производительность растёт при переходе от 1 к 2 потокам, затем деградирует при дальнейшем увеличении числа потоков. Минимальное реальное время для SimpleThreadPool достигается при 2 потоках (3,76 мс), для WorkStealingPool — при 1 потоке (4,39 мс). При 16 потоках SimpleThreadPool показывает реальное время 12,5 мс,

Таблица 5 – Результаты сценария 2: неравномерная нагрузка (10 000 задач, половина длительностью 200 мкс)

Потоков	Реализация	Time, мс	CPU, мс	items/s
1	SimpleThreadPool	1268	35,9	7 884
1	WorkStealingPool	1266	2,97	7 896
2	SimpleThreadPool	638	24,8	15 671
2	WorkStealingPool	635	16,9	15 755
4	SimpleThreadPool	317	15,5	31 542
4	WorkStealingPool	317	7,65	31 588
8	SimpleThreadPool	159	11,2	63 088
8	WorkStealingPool	237	3,50	42 228
16	SimpleThreadPool	80,3	10,4	124 480
16	WorkStealingPool	98,1	2,75	101 895

WorkStealingPool — 15,2 мс.

В сценарии 2 наблюдается принципиальное различие между метриками реального и процессорного времени. При 16 потоках SimpleThreadPool демонстрирует реальное время 80,3 мс при процессорном времени 10,4 мс, тогда как WorkStealingPool — реальное время 98,1 мс при процессорном времени 2,75 мс. Таким образом, WorkStealingPool потребляет в 3,8 раза меньше процессорного времени, однако выполняет бенчмарк в 1,2 раза медленнее по реальному времени. Данное соотношение метрик свидетельствует о принципиальном различии в стратегии ожидания задач между двумя реализациями, что будет подробно рассмотрено в разделе 3.3.

Для наглядного сравнения масштабируемости обеих реализаций в таблице 6 представлены значения ускорения $S_P = T_1/T_P$ относительно однопоточного запуска для каждого сценария.

Таблица 6 – Ускорение S_P относительно однопоточного запуска

Потоков	Simple, сц. 1	WS, сц. 1	Simple, сц. 2	WS, сц. 2
1	1,00	1,00	1,00	1,00
2	1,20	0,90	1,99	1,99
4	0,67	0,55	4,00	3,99
8	0,32	0,30	7,97	5,34
16	0,36	0,29	15,79	12,90

Из таблицы 6 следует что в сценарии 1 обе реализации не обеспечивают ускорения при увеличении числа потоков — напротив, производительность

снижается. В сценарии 2 обе реализации демонстрируют близкое к линейному ускорение при малом числе потоков (до 4 включительно), однако при 8 и 16 потоках `WorkStealingPool` демонстрирует меньшее ускорение чем `SimpleThreadPool`.

3.3 Тестирование и анализ результатов

3.3.1 Тестирование корректности

Тестирование корректности библиотеки осуществляется посредством автоматических тестов, построенных на основе библиотеки Google Test. Тесты организованы в три независимых набора, соответствующих трём модулям библиотеки. Все тесты запускаются с включённым Thread Sanitizer, что позволяет обнаруживать гонки данных во время выполнения.

Набор тестов для `WorkStealingDeque` содержит 12 тестов, разделённых на две группы. Первая группа проверяет корректность операций в однопоточном режиме, вторая — корректность при конкурентном доступе нескольких потоков. Описание тестов представлено в таблице 7.

Тест `SingleElementRaceCondition` заслуживает отдельного внимания. Он многократно (10 000 итераций) создаёт очередь с одним элементом и запускает одновременно владельца и вора, каждый из которых пытается забрать этот элемент. После завершения обоих потоков проверяется что задача была выполнена ровно один раз. Данный тест выявляет ошибки в обработке граничного случая, когда владелец и вор одновременно претендуют на последний элемент очереди.

Набор тестов для `SimpleThreadPool` содержит 6 тестов, проверяющих основные аспекты работы пула. Описание тестов представлено в таблице 8.

Набор тестов для `WorkStealingPool` содержит 10 тестов. По структуре он аналогичен набору для `SimpleThreadPool`, однако дополнительно включает тесты специфичные для механизма `work-stealing`. Описание тестов представлено в таблице 9.

Результаты запуска тестов подтверждают корректность всех реализованных компонентов: все 28 тестов из трёх наборов успешно пройдены. Отсутствие сообщений об ошибках со стороны Thread Sanitizer подтверждает корректность синхронизации в реализованных структурах данных и отсутствие гонок данных при конкурентном доступе нескольких потоков.

Таблица 7 – Тесты корректности WorkStealingDeque

Тест	Что проверяется
InitiallyEmpty	Очередь после создания пуста.
PushIncreasesSize	Размер очереди увеличивается после push.
PopOnEmptyReturnsFalse	pop возвращает false на пустой очереди.
StealOnEmptyReturnsFalse	steal возвращает false на пустой очереди.
PopIsLIFO	Владелец извлекает задачи в порядке LIFO.
StealIsFIFO	Вор извлекает задачи в порядке FIFO.
PopDecreasesSize	Размер очереди уменьшается после pop.
GrowsWhenFull	Очередь корректно расширяется при переполнении.
SingleThiefStealsAll	Один вор корректно крадёт все задачи.
OwnerAndThiefConcurrently	Владелец и вор работают одновременно, каждая задача выполняется ровно один раз.
MultipleThievesConcurrently	Несколько воров работают одновременно, каждая задача выполняется ровно один раз.
SingleElementRaceCondition	Граничный случай: один элемент в очереди, гонка между владельцем и вором.

3.3.2 Анализ результатов бенчмарков

Анализ результатов измерений, представленных в разделе 3.2.3, позволяет сформулировать ряд выводов о масштабируемости и эффективности обеих реализаций пула потоков при различных сценариях нагрузки.

В сценарии 1 (много мелких задач) обе реализации демонстрируют схожую и неожиданную тенденцию: производительность не растёт с увеличением числа потоков, а деградирует. Оптимальная пропускная способность для SimpleThreadPool достигается при 2 потоках (2 659 860 задач/с), тогда как при 16 потоках она падает до 802 883 задач/с —

Таблица 8 – Тесты корректности SimpleThreadPool

Тест	Что проверяется
ReturnsCorrectResult	Задача выполняется и возвращает корректный результат через <code>std::future</code> .
MultipleTasksExecuted	100 задач выполняются и возвращают корректные результаты.
AllTasksAreExecuted	1 000 задач выполняются все без исключения.
ExceptionPropagated	Исключение внутри задачи корректно пробрасывается через <code>std::future::get()</code> .
SingleThreadSequential	При одном рабочем потоке задачи выполняются в порядке постановки.
ThreadCountIsCorrect	Метод <code>thread_count()</code> возвращает корректное количество созданных потоков.

снижение в 3,3 раза. Для `WorkStealingPool` оптимум достигается при 1 потоке (2 275 520 задач/с), а при 16 потоках падает до 656 266 задач/с — снижение в 3,5 раза.

Данный результат объясняется соотношением времени выполнения задачи и накладных расходов планировщика. Атомарный инкремент выполняется за единицы наносекунд, тогда как постановка задачи в пул включает создание `std::packaged_task`, `std::future` и захват мьютекса — операции, занимающие сотни наносекунд. При большом числе потоков конкуренция за разделяемые структуры данных усиливается, что приводит к дальнейшему росту накладных расходов. Таким образом, для задач с временем выполнения порядка наносекунд использование пула потоков вообще нецелесообразно: накладные расходы планировщика многократно превышают полезную работу.

Сравнение двух реализаций в сценарии 1 показывает что `SimpleThreadPool` незначительно превосходит `WorkStealingPool` при всех значениях числа потоков. Разница составляет от 3% при 1 потоке до 18% при 16 потоках. Данный результат согласуется с ожиданиями: при мелких задачах механизм кражи не даёт преимущества, поскольку задачи выполняются быстрее чем успевает сработать механизм кражи, а мьютекс в `WorkStealingDeque` вносит дополнительные накладные расходы по

Таблица 9 – Тесты корректности WorkStealingPool

Тест	Что проверяется
ReturnsCorrectResult	Задача выполняется и возвращает корректный результат через <code>std::future</code> .
MultipleTasksExecuted	100 задач возвращают корректные результаты.
AllTasksAreExecuted	1 000 задач выполняются все без исключения.
ExceptionPropagated	Исключение пробрасывается через <code>std::future</code> .
ThreadCountIsCorrect	Метод <code>thread_count()</code> возвращает корректное число рабочих потоков.
SingleThreadExecutes	Пул с одним потоком выполняет все задачи корректно.
HeavyLoadAllTasks	100 000 задач выполняются корректно под высокой нагрузкой.
UnevenTaskDurations	Все задачи выполняются при неравномерной нагрузке.
ResultsUnderContention	Корректность результатов при конкуренции потоков.
DestructorWaitsPending	Деструктор дожидается завершения всех задач.

сравнению с единой глобальной очередью `SimpleThreadPool`.

В сценарии 2 (неравномерная нагрузка) картина принципиально иная. При малом числе потоков (1, 2, 4) обе реализации демонстрируют практически идентичное реальное время выполнения и близкое к линейному ускорение: при 4 потоках обе реализации ускоряются примерно в 4 раза по сравнению с однопоточным запуском. Это объясняется тем что при малом числе потоков нагрузка распределяется равномерно и механизм кражи не оказывает существенного влияния.

При 8 и 16 потоках картина меняется. `SimpleThreadPool` при 16 потоках демонстрирует реальное время 80,3 мс и ускорение 15,79 относительно однопоточного запуска. `WorkStealingPool` при тех же условиях показывает реальное время 98,1 мс и ускорение 12,90. Таким образом, вопреки теоретическим ожиданиям, `SimpleThreadPool` оказывается быстрее в данном сценарии.

Ключом к пониманию данного результата является сравнение метрик реального и процессорного времени. При 16 потоках `SimpleThreadPool` потребляет 10,4 мс процессорного времени при реальном времени 80,3 мс. `WorkStealingPool` потребляет лишь 2,75 мс процессорного времени при реальном времени 98,1 мс. Соотношение реального и процессорного времени составляет 7,7 для `SimpleThreadPool` и 35,7 для `WorkStealingPool`. Столь высокое соотношение для `WorkStealingPool` означает что потоки подавляющую часть времени проводят не в выполнении задач, а в состоянии ожидания.

Данный эффект обусловлен механизмом ожидания в `WorkStealingPool`: при отсутствии задач поток засыпает на время до 1 мс посредством `cv_.wait_for()`. В сценарии неравномерной нагрузки при большом числе потоков значительная часть потоков быстро выполняет короткие задачи и переходит в состояние ожидания, пока другие потоки заняты длинными задачами. Пробуждение спящих потоков происходит с задержкой до 1 мс, что увеличивает реальное время выполнения. `SimpleThreadPool` лишён данного недостатка: его потоки ожидают на `condition_variable` и пробуждаются немедленно при появлении новой задачи в общей очереди.

Таким образом, результаты экспериментального исследования выявляют фундаментальный компромисс между двумя реализациями. `SimpleThreadPool` обеспечивает меньшую латентность реакции на новые задачи за счёт более высокого потребления процессорного времени. `WorkStealingPool` экономит процессорное время за счёт более высокой латентности ожидания, что при данной реализации механизма ожидания нивелирует теоретические преимущества механизма кражи задач.

3.3.3 Обсуждение результатов

Результаты экспериментального исследования не подтвердили исходную гипотезу о превосходстве `WorkStealingPool` над `SimpleThreadPool` по показателю реального времени выполнения. Данный результат требует детального обсуждения, поскольку он не противоречит теории work-stealing, а отражает ограничения конкретной реализации.

Теоретическое превосходство механизма work-stealing, доказанное

Блюмофе и Лейзерсоном [3], предполагает два условия. Во-первых, операции владельца над локальной очередью должны выполняться без блокирующей синхронизации — именно это обеспечивает низкую конкуренцию между потоками. Во-вторых, задачи должны иметь достаточно большое время выполнения чтобы накладные расходы на кражу были пренебрежимо малы по сравнению с полезной работой.

В разрабатываемой библиотеке оба условия нарушены. Первое условие нарушено вследствие использования `std::mutex` в `WorkStealingDeque`: все три операции — `push`, `pop` и `steal` — являются блокирующими. Это означает что при конкурентном доступе владельца и вора к очереди возникает конкуренция за мьютекс, аналогичная конкуренции за глобальную очередь в `SimpleThreadPool`. Принципиальное преимущество lock-free реализации deque — отсутствие блокировок при операциях владельца в типичном случае — в данной реализации не достигается.

Второе условие нарушено в сценарии 1, где время выполнения задачи (атомарный инкремент, ≈ 5 нс) на несколько порядков меньше накладных расходов планировщика (сотни наносекунд на создание `std::packaged_task` и захват мьютекса). В таких условиях ни одна реализация пула потоков не способна обеспечить масштабирование: доля накладных расходов планировщика в общем времени выполнения стремится к единице, что по закону Амдала [4] ограничивает ускорение значением, близким к единице.

Влияние мьютекса в `WorkStealingDeque` проявляется двояко. С одной стороны, мьютекс устраняет преимущество локальных очередей: вместо того чтобы работать с локальными данными без синхронизации, владелец вынужден захватывать мьютекс при каждой операции. Это делает локальную очередь `WorkStealingPool` не быстрее, а медленнее глобальной очереди `SimpleThreadPool` при высокой интенсивности операций — ведь в `SimpleThreadPool` один мьютекс делится между всеми потоками и захватывается реже чем P локальных мьютексов в `WorkStealingPool`. С другой стороны, мьютекс обеспечивает корректность реализации, что является необходимым условием для любых измерений производительности.

Компромисс между латентностью и энергоэффективностью является важным практическим результатом исследования. В сценарии 2 при

16 потоках `WorkStealingPool` потребляет в 3,8 раза меньше процессорного времени чем `SimpleThreadPool` (2,75 мс против 10,4 мс), однако проигрывает по реальному времени выполнения (98,1 мс против 80,3 мс). Данный компромисс обусловлен механизмом ожидания: потоки `WorkStealingPool` засыпают на время до 1 мс при отсутствии задач, тогда как потоки `SimpleThreadPool` пробуждаются мгновенно через `condition_variable`.

С практической точки зрения данный компромисс определяет условия применимости каждой реализации. `SimpleThreadPool` предпочтителен в системах с жёсткими требованиями к латентности обработки запросов — в частности, в высоконагруженных серверных приложениях, где каждая миллисекунда задержки влияет на пользовательский опыт. `WorkStealingPool` в текущей реализации предпочтителен в системах где энергоэффективность важнее латентности — например, в фоновых вычислительных задачах на мобильных устройствах или в системах с ограниченным энергопотреблением. Следует также отметить что механизм ожидания `WorkStealingPool` может быть настроен: уменьшение таймаута `cv_.wait_for()` снизит латентность за счёт роста потребления процессорного времени, а увеличение — наоборот. Данная гибкость является преимуществом разработанной библиотеки перед промышленными решениями с фиксированными параметрами планировщика.

Таким образом, полученные результаты свидетельствуют о том что выбор реализации пула потоков должен определяться конкретными требованиями приложения, а не общими соображениями о теоретической эффективности того или иного алгоритма.

Выводы по главе 3

В ходе программной реализации и экспериментального исследования были получены следующие выводы.

Во-первых, практическая реализация библиотеки потребовала решения ряда нетривиальных проблем, не очевидных на этапе проектирования. Гонка данных в деструкторе `SimpleThreadPool`, переполнение беззнакового типа в методе `pop()` и трудноустраняемые ошибки в lock-free реализации `WorkStealingDeque` были выявлены в ходе тестирования с Thread Sanitizer и исправлены до проведения измерений производительности. Все 28 автоматических тестов корректности успешно пройдены.

Во-вторых, результаты экспериментального исследования не подтвердили гипотезу о превосходстве `WorkStealingPool` по показателю реального времени выполнения. Основной причиной является использование блокирующей синхронизации в `WorkStealingDeque`, что нивелирует теоретическое преимущество локальных очередей. Данный результат подчёркивает принципиальную важность корректной реализации неблокирующей двусторонней очереди для достижения теоретически предсказанной эффективности механизма work-stealing.

В-третьих, исследование выявило фундаментальный компромисс между латентностью и энергоэффективностью двух реализаций. `SimpleThreadPool` обеспечивает меньшую латентность реакции на новые задачи, что делает его предпочтительным для систем с жёсткими временными требованиями. `WorkStealingPool` потребляет значительно меньше процессорного времени при неравномерной нагрузке, что делает его предпочтительным для фоновых вычислений в системах с ограниченным энергопотреблением.

ЗАКЛЮЧЕНИЕ

В настоящей выпускной квалификационной работе была поставлена цель разработать библиотеку на языке C++, реализующую пул потоков с механизмом work-stealing, и экспериментально исследовать её производительность в сравнении с классическим пулом потоков при различных сценариях нагрузки. Поставленная цель достигнута, все задачи решены. Ниже приведены основные результаты работы.

В ходе анализа предметной области установлено, что переход к многоядерным архитектурам сделал параллелизм неотъемлемым требованием к современному программному обеспечению. Классические подходы к управлению потоками — модель «поток на задачу» и пул потоков с единой глобальной очередью — обнаруживают существенные ограничения с точки зрения масштабируемости и эффективности балансировки нагрузки при большом числе ядер. Механизм work-stealing обладает строгим теоретическим обоснованием и доказанной оптимальностью по времени выполнения и потреблению памяти, однако существующие промышленные реализации — Intel TBB и Microsoft PPL — не допускают лёгкого изучения и модификации, а стандартная библиотека C++ не предоставляет готовой реализации пула потоков с work-stealing. Данное обстоятельство обусловило актуальность настоящего исследования.

В ходе теоретического обоснования и проектирования решения формальная модель параллельных вычислений на основе ориентированного ациклического графа позволила строго обосновать эффективность механизма work-stealing. Доказанная Блюмофе и Лейзерсоном оценка $T_P \leq T_1/P + O(T_\infty)$ показывает, что алгоритм work-stealing обеспечивает время выполнения, близкое к теоретическому минимуму, без каких-либо априорных знаний о структуре вычислений. Закон Амдала определил ключевое требование к реализации: накладные расходы на управление очередями задач должны быть минимизированы, поскольку они увеличивают эффективную долю последовательной части вычисления и ограничивают достижимое ускорение. Спроектированная архитектура библиотеки включает два модуля с идентичным интерфейсом метода `submit()`: классический пул потоков `SimpleThreadPool` и пул потоков с механизмом work-stealing `WorkStealingPool`. Идентичность интерфейса обеспечивает

взаимозаменяемость реализаций и корректность сравнительного исследования. Выбор упрощённой реализации `WorkStealingDeque` на основе `std::deque` и `std::mutex` вместо неблокирующего алгоритма Чейза-Лева является обоснованным компромиссом: корректная реализация неблокирующей очереди требует применения механизмов безопасного освобождения памяти в конкурентной среде, реализация которых выходила за рамки настоящей работы.

В ходе программной реализации и экспериментального исследования был получен ряд практически значимых результатов. Практическая реализация библиотеки потребовала решения нетривиальных проблем, не очевидных на этапе проектирования: гонка данных в деструкторе `SimpleThreadPool`, переполнение беззнакового типа в методе `pop()` и трудноустраняемые ошибки в lock-free реализации `WorkStealingDeque` были выявлены с помощью инструмента Thread Sanitizer и исправлены до проведения измерений производительности. Все 28 автоматических тестов корректности успешно пройдены, гонок данных не обнаружено.

Результаты экспериментального исследования не подтвердили исходную гипотезу о превосходстве `WorkStealingPool` по показателю реального времени выполнения. Основной причиной является использование блокирующей синхронизации в `WorkStealingDeque`, что нивелирует теоретическое преимущество локальных очередей и подчёркивает принципиальную важность корректной реализации неблокирующей двусторонней очереди для достижения теоретически предсказанной эффективности механизма work-stealing. Вместе с тем исследование выявило фундаментальный компромисс между латентностью и энергоэффективностью двух реализаций. `SimpleThreadPool` обеспечивает меньшую латентность реакции на новые задачи, что делает его предпочтительным для систем с жёсткими временными требованиями. `WorkStealingPool` потребляет в 3,8 раза меньше процессорного времени при неравномерной нагрузке — при 16 потоках 2,75 мс против 10,4 мс — что делает его предпочтительным для фоновых вычислений в системах с ограниченным энергопотреблением. Таким образом, выбор реализации пула потоков должен определяться конкретными требованиями приложения, а не общими соображениями о теоретической эффективности того или иного алгоритма.

Перспективами дальнейшего развития данной работы являются

следующие направления. Во-первых, реализация неблокирующей двусторонней очереди по алгоритму Чейза-Лева с применением механизма hazard pointers или эпохального удаления позволит устранить главное ограничение текущей реализации и проверить теоретические гарантии алгоритма work-stealing в условиях отсутствия блокирующей синхронизации. Во-вторых, исследование влияния топологии памяти NUMA на производительность планировщика представляет практический интерес для многопроцессорных серверных систем: привязка каждого рабочего потока к конкретному NUMA-узлу и ограничение кражи задач в пределах одного узла способны существенно снизить латентность доступа к памяти. В-третьих, адаптивное управление числом рабочих потоков в зависимости от текущей загрузки системы позволит библиотеке автоматически подстраиваться под изменяющийся характер нагрузки без ручной настройки параметров. Наконец, расширение набора сценариев нагрузки за счёт рекурсивных алгоритмов типа «разделяй и властвуй» — сортировки слиянием, параллельного обхода деревьев — позволит исследовать механизм work-stealing в тех условиях, для которых он изначально проектировался и в которых его теоретические преимущества должны проявляться наиболее полно.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Sutter H. The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software // Dr. Dobbs's Journal. — 2005. — Т. 30, № 3. — С. 202—210.
2. Williams A. C++ Concurrency in Action. — 2-е изд. — Shelter Island, NY : Manning Publications, 2019.
3. Blumofe R. D., Leiserson C. E. Scheduling Multithreaded Computations by Work Stealing // Journal of the ACM. — 1999. — Т. 46, № 5. — С. 720—748. — DOI: 10.1145/324133.324234.
4. Amdahl G. M. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities // Proceedings of the April 18–20, 1967, Spring Joint Computer Conference. — 1967. — С. 483—485. — DOI: 10.1145/1465482.1465560.
5. Chase D., Lev Y. Dynamic Circular Work-Stealing Deque // Proceedings of the 17th Annual ACM Symposium on Parallelism in Algorithms and Architectures. — 2005. — С. 21—28. — DOI: 10.1145/1073970.1073974.
6. Intel Corporation. Intel® Xeon® Scalable Processors. — 2023. — URL: <https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable.html> (дата обращения 01.11.2025).
7. Lameter C. NUMA (Non-Uniform Memory Access): An Overview // ACM Queue. — 2013. — Т. 11, № 7. — С. 40—51. — DOI: 10.1145/2508834.2513149.
8. The Art of Multiprocessor Programming / М. Herlihy [и др.]. — 2-е изд. — Cambridge, MA : Morgan Kaufmann, 2020.
9. Drepper U., Molnar I. The Native POSIX Thread Library for Linux. — Red Hat Inc., 2005.
10. Butenhof D. R. Programming with POSIX Threads. — Boston, MA : Addison-Wesley, 1997.
11. Love R. Linux Kernel Development. — 3-е изд. — Boston, MA : Addison-Wesley, 2010.
12. Tanenbaum A. S., Bos H. Modern Operating Systems. — 4-е изд. — Hoboken, NJ : Pearson, 2014.

13. Reactive Streams Working Group. Reactive Streams Specification. — 2014. — URL: <https://www.reactive-streams.org> (дата обращения 07.12.2025).
14. Drepper U. What Every Programmer Should Know About Memory. — Red Hat Inc., 2007.
15. Michael M. M., Scott M. L. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms // Proceedings of the 15th Annual ACM Symposium on Principles of Distributed Computing. — 1996. — С. 267—275. — DOI: 10.1145/248052.248106.
16. Acar U. A., Blleloch G. E., Blumofe R. D. The Data Locality of Work Stealing // Proceedings of the 12th Annual ACM Symposium on Parallel Algorithms and Architectures. — 2000. — С. 1—12. — DOI: 10.1145/341800.341801.
17. Reeves G. E. What Really Happened on Mars? // IEEE Spectrum. — 1998. — Т. 35, № 2. — С. 18—25.
18. Wiggins A. The Twelve-Factor App. — 2012. — URL: <https://12factor.net> (дата обращения 15.12.2025).
19. Welsh M., Culler D., Brewer E. SEDA: An Architecture for Well-Conditioned, Scalable Internet Services // ACM SIGOPS Operating Systems Review. — 2001. — Т. 35, № 5. — С. 230—243. — DOI: 10.1145/502059.502057.
20. The Worst-Case Execution-Time Problem — Overview of Methods and Survey of Tools / R. Wilhelm, J. Engblom, A. Ermedahl [и др.] // ACM Transactions on Embedded Computing Systems. — 2008. — Т. 7, № 3. — С. 1—53. — DOI: 10.1145/1347375.1347389.
21. An EDF Scheduling Class for the Linux Kernel / D. Faggioli [и др.] // Proceedings of the 11th Real-Time Linux Workshop. — 2009. — С. 1—10.
22. Dongarra J., Heroux M. A., Luszczek P. High-Performance Conjugate-Gradient Benchmark: A New Metric for Ranking High-Performance Computing Systems // The International Journal of High Performance Computing Applications. — 2016. — Т. 30, № 1. — С. 3—10. — DOI: 10.1177/1094342015593158.
23. Halstead R. H. Multilisp: A Language for Concurrent Symbolic Computation // ACM Transactions on Programming Languages and Systems. — 1985. — Т. 7, № 4. — С. 501—538. — DOI: 10.1145/4472.4478.

ПРИЛОЖЕНИЕ А

Исходный код

Полный исходный код разработанного программного обеспечения размещён в репозитории GitHub. Переход к нему осуществляется по ссылке, зашифрованной в QR-коде на рисунке А.1.



Рисунок А.1 – QR-код для доступа к репозиторию